Infinite State Model Checking by Learning Transitive Relations

Florian Frohn and Jürgen Giesl

Programming Languages and Verification, RWTH Aachen University.

Contributing authors: florian.frohn@cs.rwth-aachen.de;
giesl@informatik.rwth-aachen.de;

Abstract

We propose a new approach for proving safety of infinite state systems. It extends the analyzed system by *transitive relations* until its *diameter* $\boldsymbol{D}$ becomes finite, i.e., until constantly many steps suffice to cover all reachable states, irrespective of the initial state. Then we can prove safety by checking that no error state is reachable in $\boldsymbol{D}$ steps. To deduce transitive relations, we use *recurrence analysis*. While recurrence analyses can usually find conjunctive relations only, our approach also discovers disjunctive relations by combining recurrence analysis with *projections*. If our approach fails to prove safety, then we obtain a potential counterexample that may contain learned relations which resulted from our recurrence analysis. To prove unsafety, we show how to check whether such examples are indeed counterexamples by replacing learned relations with relations resulting from *acceleration techniques*. An empirical evaluation of the implementation of our approach in our tool LoAT shows that it is highly competitive with the state of the art.

1 Introduction

We consider relations defined by SMT formulas over two disjoint vectors of *pre-* and *post-variables* $\mathbf{x}$ and $\mathbf{x}'$. Such *relational formulas* can easily represent *transition systems* (TSs), linear *Constrained Horn Clauses* (CHCs), and *control-flow automata* (CFAs).[1] Thus, they subsume many popular intermediate representations used for verification of systems specified in more expressive languages.

[1]To this end, it suffices to introduce one additional variable that represents the control-flow location (for TSs and CFAs) or the predicate (for linear CHCs).

In contrast to, e.g., source code, relational formulas are unstructured. However, source code may be unstructured, too (e.g., due to `gotos`), so being independent from the structure of the input makes our approach broadly applicable.

Example 1 (Running Example)**.** Let $\tau := \tau_{\mathsf{inc}} \vee \tau_{\mathsf{dec}}$ with:

$$w \doteq 0 \wedge x' \doteq x + 1 \wedge y' \doteq y + 1 \qquad (\tau_{\mathsf{inc}})$$
$$w' \doteq w \wedge w \doteq 1 \wedge x' \doteq x - 1 \wedge y' \doteq y - 1 \qquad (\tau_{\mathsf{dec}})$$

We use "$\doteq$" for equality in relational formulas. The formula τ defines a relation $\to_\tau$ on $\mathbb{Z} \times \mathbb{Z} \times \mathbb{Z}$ by relating the non-primed pre-variables with the primed post-variables. So for all $v_w, v_x, v_y, v'_w, v'_x, v'_y \in \mathbb{Z}$, we have $(v_w, v_x, v_y) \to_\tau (v'_w, v'_x, v'_y)$ iff $[w/v_w, x/v_x, y/v_y, w'/v'_w, x'/v'_x, y'/v'_y]$ is a model of τ. Let the set of *error states* be given by $\psi_{\mathsf{err}} := w \doteq 1 \wedge x \leq 0 \wedge y > 0$.

With the *initial states* $\psi_{\mathsf{init}} := x \doteq 0 \wedge y \doteq 0$ this example[2] is challenging for existing model checkers: Neither the default configuration of Z3/Spacer [41,51], nor Golem's [6] implementation of Spacer, LAWI [50], IMC [49], TPA [10], PDKIND [38], or predicate abstraction [33] can prove its safety.[3] In contrast, all of these techniques can prove safety with the more general initial states $\psi_{\mathsf{init}} := x \doteq y$. As all of them are based on *interpolation*, the reason might be that the inductive invariant $x \doteq y$ is now a subterm of ψ_{init}, so it is likely to occur in interpolants. However, this explanation is insufficient, as all techniques fail again for $\psi_{\mathsf{init}} := x \doteq y \wedge y \doteq 0$.

This illustrates a well-known issue of interpolation-based verification techniques: They are highly sensitive to minor changes of the input or the underlying interpolating SMT solver (e.g., [43, p. 102]). So while they can often solve difficult problems quickly, they sometimes fail for easy examples like the one above.

In another line of research, *recurrence analysis* has been used for software verification [20,40]. Here, the idea is to extract recurrence equations from loops and solve them to summarize the effect of arbitrarily many iterations. While recurrence analysis often yields very precise summaries for loops without branching, these summaries are conjunctive. However, for loops with branching, disjunctive summaries are often important to distinguish the branches.

In this work, we embed recurrence analysis into *bounded model checking* (BMC) [5], resulting in a robust, competitive model checking algorithm. To find disjunctive summaries, we exploit the structure of the relational formula to partition the state space on the fly via *model based projection* [41] (which approximates quantifier elimination), and a variation of recurrence analysis called *transitive projection*.

Our approach is inspired by ABMC [26], which combines BMC with *acceleration* [7,21,42]. ABMC uses *blocking clauses* to speed up the search for counterexamples, but they turned out to be of little use for this purpose. Instead, they enable ABMC to prove safety of certain challenging benchmarks. This motivated the development of

[2] `extra-small-lia/bouncy_symmetry` from the CHC competition
[3] To test that, we used the same versions of these tools as for our empirical evaluation in Sect. 8.

Algorithm 1: TRL (high level)

Input: initial states, relational formula τ, and error states
Output: safe or unknown

```
1  encode(initial states)
2  while ⊤
3      encode(error states)
4      if check_sat() do return unknown
5      remove the encoding of the error states
6      encode(learned relations must not be used twice in a row)
7      encode(one more unrolling of →_τ and of all learned relations)
8      if ¬check_sat() do return safe
9      σ ← model();   let v_1 →_{τ_1} ... →_{τ_{k-1}} v_k be the run that corresponds to σ
10     if v_s →_{τ_s} ... →_{τ_{s+ℓ-1}} v_{s+ℓ} is a loop do
11         if v_s ↛_π v_{s+ℓ} for all learned relations →_π do
12             learn a transitive relation from the loop and σ
13         pick learned relation →_π with v_s →_π v_{s+ℓ}
14         encode(→_π has preference over loops from v_s to v_{s+ℓ})
15         backtrack to the point where →_τ was only unrolled s − 1 times
```

our novel dedicated algorithm for proving safety via BMC and blocking clauses. See Sect. 7 for a detailed comparison with ABMC.

Overview

We start with an informal explanation of our approach. Given a relational formula τ, one can prove safety with BMC by unrolling its transition relation $\to_\tau$ D times, where D is the *diameter* [5]. So D is the "longest shortest path" from an initial to some other state, or more formally:

$$D := \sup_{\mathbf{v}' \text{ is reachable from an initial state}} \left(\min\{i \in \mathbb{N} \mid \mathbf{v} \text{ is an initial state}, \mathbf{v} \to_\tau^i \mathbf{v}'\}\right)$$

So every reachable state can be reached in $\leq D$ steps. Hence, if BMC finds no counterexample in D steps, the system is safe. Alg. 1 is a high level version of our new algorithm *Transitive Relation Learning* (TRL), where this observation is exploited in Line 8, as unsatisfiability of the underlying SMT problem implies that the diameter has been reached (see the end of this section for more details). For infinite state systems, D is rarely finite: Consider the relational formula

$$x' \doteq x + 1 \tag{1}$$

with the initial state $x \doteq 0$. Then $k \in \mathbb{N}$ steps are needed to reach a state with $x \doteq k$. So D is infinite, i.e., the diameter cannot be used to prove safety directly.

The core idea of TRL is to "enlarge" τ to decrease its diameter (even though TRL never computes the diameter explicitly). To this end, our approach "adds" transitive

relations to τ, which will be called *learned relations* in the sequel (Line 12). For (1), TRL would learn the relational formula

$$n > 0 \wedge x' \doteq x + n. \tag{2}$$

Then the diameter of (1) $\vee$ (2) is 1, as a state with $x \doteq k$ can be reached in 1 step by setting n to k. Transitive relations are particularly suitable for decreasing the diameter, as they allow us to ignore runs where the same learned relation is used twice in a row. This is exploited in Line 6 of Alg. 1, where $\mathsf{encode}(P)$ means that we add an encoding of the property P to the underlying SMT problem.

This raises the questions *when* and *how* new relations should be learned. Regarding the "when", our approach unrolls the transition relation, just like BMC (Line 7). Thereby, it looks for *loops* (Line 10) and learns a relation when a new loop is encountered (Line 12). However, as we analyze unstructured systems, the definition of a "loop" is not obvious. Details will be discussed in Sect. 3.2.

Regarding the "how", TRL ensures that we have a *model* for the loop, i.e., a valuation σ_{loop} that corresponds to an evaluation of the loop body (which can be extracted from σ in Alg. 1). Then given a loop τ_{loop} and a model σ_{loop}, apart from transitivity we only require two more properties for a learned relation $\rightarrow_\pi$. First, the evaluation that corresponds to σ_{loop} must also be possible with $\rightarrow_\pi$. This ensures that Alg. 1 makes "progress", i.e., that we do not unroll the same loop with the same model again. Second, the following set must be finite:

$$\{\pi \mid \sigma_{\mathsf{loop}} \text{ is a model of } \tau_{\mathsf{loop}}, \rightarrow_\pi \text{ is learned from } \tau_{\mathsf{loop}} \text{ and } \sigma_{\mathsf{loop}}\}$$

So TRL only learns finitely many relations from a given loop τ_{loop}. While TRL may diverge (Remark 23), this "usually" ensures termination in practice.

Apart from these restrictions, we have lots of freedom when computing learned relations, as "enlarging" τ (i.e., adding disjuncts to τ) is *always* sound for proving safety. The *transitive projection* that we use to learn relations (see Sect. 4) heavily exploits this freedom. It modifies the recurrence analysis from [20] by replacing expensive operations – convex hulls and polyhedral projections – by a cheap variation of *model based projection* [41]. While convex hulls and polyhedral projections are over-approximations (for integer arithmetic), model based projections under-approximate. This is surprising at first, but the justification for using under-approximations is that, as mentioned above, "enlarging" τ is always sound.

Without our modifications, recurrence analysis over-approximates, so we "mix" over- and under-approximations. Thus, learned relations are *not* under-approximations, so Alg. 1 cannot prove unsafety and returns $\mathsf{unknown}$ in Line 4. We show how to adapt our approach for also proving unsafety in Sect. 5.

Learned relations may reduce the diameter, but computing the diameter is difficult. Instead, TRL adds *blocking clauses* to the SMT encoding that force the SMT solver to prefer learned relations over loops (Line 14). Then unsatisfiability implies that the diameter has been reached, so that safe can be returned (Line 8).

For our example (1), once (2) has been learned, it is preferred over (1). As (2) must not be used twice in a row (Line 6), the SMT problem in Line 8 becomes unsatisfiable

after adding a blocking clause for $s = \ell = 1$ (that blocks (1) for the 1st step), and one for $s = 2$ and $\ell = 1$ (that blocks (1) for the 2nd step). Since we check for reachability of error states after every step (Line 4), this implies safety.

Outline

After introducing preliminaries in Sect. 2, we present our new algorithm TRL in Sect. 3. As TRL builds upon *transitive projections*, we show how to implement such a projection for linear integer arithmetic in Sect. 4. In Sect. 5, we adapt TRL to prove unsafety, and in Sect. 6, we explain how to improve our approach by also considering backward-reachability (i.e., whether an initial state is backward-reachable from an error state). Sect. 7 discusses related work and we evaluate our approach empirically in Sect. 8. All missing proofs can be found in App. A.

A preliminary version of this paper appeared at CADE '25 [27]. The present paper extends [27] substantially:

- We adapt our approach in order to also prove unsafety in Sect. 5.
- We improve our approach by considering backward-reachability in Sect. 6.
- We illustrate in detail how our approach solves our leading example in Sect. 3.5.
- We extend our empirical evaluation in Sect. 8 in two ways:
  - We include 7 additional configurations of various tools in order to compare our approach with more competing techniques.
  - We include 2 more configurations of TRL to illustrate the effect of considering backward-reachability.
- We discuss related work in more detail in Sect. 7.
- We include all proofs that were missing in [27].
- We present many additional explanations.

2 Preliminaries

We assume familiarity with basics of first-order logic [17]. $\mathcal{V}$ is a countably infinite set of variables and $\mathcal{A}$ is a first-order theory with signature Σ and carrier $\mathcal{C}$. For each entity e, $\mathcal{V}(e)$ is the set of variables that occur in e. $\mathsf{QF}(\Sigma)$ denotes the set of all quantifier-free first-order formulas over Σ, and $\mathsf{QF}_\wedge(\Sigma)$ only contains conjunctions of Σ-literals. $\top$ and $\bot$ stand for "true" and "false", respectively.

Given $\psi \in \mathsf{QF}(\Sigma)$ with $\mathcal{V}(\psi) = \mathbf{y}$, we say that ψ is $\mathcal{A}$-*valid* (written $\models_\mathcal{A} \psi$) if every model of $\mathcal{A}$ satisfies the universal closure $\forall \mathbf{y}.\ \psi$ of ψ. A partial function $\sigma : \mathcal{V} \rightharpoonup \mathcal{C}$ is called a *valuation*. If $\mathcal{V}(\psi) \subseteq \mathrm{dom}(\sigma)$ and $\models_\mathcal{A} \sigma(\psi)$, then σ is an $\mathcal{A}$-*model* of ψ (written $\sigma \models_\mathcal{A} \psi$). Here, $\sigma(\psi)$ results from ψ by instantiating all variables according to σ. If ψ has an $\mathcal{A}$-model, then ψ is $\mathcal{A}$-*satisfiable*. If $\sigma(\psi)$ is $\mathcal{A}$-satisfiable (but not necessarily $\mathcal{V}(\psi) \subseteq \mathrm{dom}(\sigma)$), then we say that ψ is $\mathcal{A}$-*consistent* with σ. We write $\psi \models_\mathcal{A} \psi'$ for $\models_\mathcal{A} \psi \implies \psi'$, and $\psi \equiv_\mathcal{A} \psi'$ means $\models_\mathcal{A} \psi \iff \psi'$. In the sequel, we omit the subscript $\mathcal{A}$, and we just say "valid", "model", "satisfiable", and "consistent". We assume that $\mathcal{A}$ is complete (i.e., $\models \psi$ or $\models \neg\psi$ holds for every closed formula over Σ) and that $\mathcal{A}$ has an effective quantifier elimination procedure (i.e., quantifier elimination is computable).

Algorithm 2: TRL

Input: a safety problem $\mathcal{T} = (\psi_{\mathsf{init}}, \tau, \psi_{\mathsf{err}})$
Output: safe or unknown

1 $b \leftarrow 0$; $\boldsymbol{\pi} \leftarrow [\tau]$; BLOCKED $\leftarrow \varnothing$
2 $\mathsf{add}(\mu_1(\psi_{\mathsf{init}}))$ // encode the initial states
3 **while** $\top$ // main loop
4 $\quad b$++; $\mathsf{push}()$; $\mathsf{add}(\mu_b(\psi_{\mathsf{err}}))$ // encode the error states
5 $\quad$ **if** check_sat() **do return** unknown **else** $\mathsf{pop}()$ // check their reachability
6 $\quad \mathsf{push}()$ // add backtracking point
7 $\quad$ **if** $b > 1$ **do** $\mathsf{add}(\overset{(b)}{x}_{\mathsf{id}} \doteq 1 \vee \overset{(b)}{x}_{\mathsf{id}} \neq \overset{(b-1)}{x}_{\mathsf{id}})$ // encode transitivity
8 $\quad \mathsf{add}(\mu_b(\bigvee_{n=1}^{|\boldsymbol{\pi}|}(\pi_n \wedge x_{\mathsf{id}} \doteq n)))$ // encode $\rightarrow_\tau$ and learned relations
9 $\quad \mathsf{add}(\bigwedge_{(b,\pi)\in \text{BLOCKED}} \pi)$ // add blocking clauses for this b
10 $\quad$ **if** $\neg$check_sat() **do return** safe // check if the search space is exhausted
11 $\quad \sigma \leftarrow \mathsf{model}()$; $\boldsymbol{\tau} \leftarrow \mathsf{trace}_b(\sigma, \boldsymbol{\pi})$ // build trace from current model
12 $\quad$ **if** $[\tau_s, \ldots, \tau_{s+\ell-1}]$ *is a loop* **do** // search loop
13 $\quad\quad \sigma_{\mathsf{loop}} \leftarrow [x/\sigma(\mu_{s,\ell}(x)) \mid x \in \mathbf{x} \cup \mathbf{x}']$ // build the valuation for the loop
14 $\quad\quad$ **if** *no* $\pi \in \mathsf{tail}(\boldsymbol{\pi})$ *is consistent with* σ_{loop} **do** // redundancy check
15 $\quad\quad\quad \tau_{\mathsf{loop}} \leftarrow \mu_{s,\ell}^{-1}(\bigwedge_{i=s}^{s+\ell-1} \mu_i(\tau_i))$ // build the loop
16 $\quad\quad\quad \boldsymbol{\pi} \leftarrow \boldsymbol{\pi} :: \mathsf{tp}(\tau_{\mathsf{loop}}, \sigma \circ \mu_{s,\ell})$ // learn relation
17 $\quad\quad$ let $\pi \in \mathsf{tail}(\boldsymbol{\pi})$ and $\overline{\sigma} \supseteq \sigma_{\mathsf{loop}}$ s.t. $\overline{\sigma} \models \pi$ // pick suitable learned relation
18 $\quad\quad$ BLOCKED.$\mathsf{add}(s + \ell - 1, \mathsf{blocking_clause}(s, \ell, \pi, \overline{\sigma}))$ // block the loop
19 $\quad\quad$ **while** $b \geq s$ { $\mathsf{pop}()$; b-- } // backtrack to the start of the loop

We write $\mathbf{x}$ for sequences and x_i is the ith element of $\mathbf{x}$, where x_1 denotes the first element. We use "::" for concatenation of sequences, where we identify sequences of length 1 with their elements, so, e.g., $x :: \mathbf{x} = [x] :: \mathbf{x}$.

Let $d \in \mathbb{N}$ be fixed, and let $\mathbf{x}, \mathbf{x}' \in \mathcal{V}^d$ be disjoint vectors of pairwise different variables, called the *pre-* and *post-variables*. All other variables are *extra variables*. Each $\tau \in \mathsf{QF}(\Sigma)$ induces a *transition relation* $\rightarrow_\tau$ on *states*, i.e., elements of $\mathcal{C}^d$, where $\mathbf{v} \rightarrow_\tau \mathbf{v}'$ iff $\tau[\mathbf{x}/\mathbf{v}, \mathbf{x}'/\mathbf{v}']$ is satisfiable. Here, $[\mathbf{x}/\mathbf{v}, \mathbf{x}'/\mathbf{v}']$ maps x_i to v_i and x_i' to v_i'.

We call $\tau \in \mathsf{QF}(\Sigma)$ a *relational formula* if we are interested in τ's induced transition relation. *Transitions* are conjunctive relational formulas without extra variables (i.e., conjunctions of literals over pre- and post-variables). We sometimes identify τ with $\rightarrow_\tau$, so we may call τ a relation.

A *τ-run* is a sequence $\mathbf{v}_1 \rightarrow_\tau \ldots \rightarrow_\tau \mathbf{v}_k$. A *safety problem* $\mathcal{T}$ is a triple $(\psi_{\mathsf{init}}, \tau, \psi_{\mathsf{err}}) \in \mathsf{QF}(\Sigma) \times \mathsf{QF}(\Sigma) \times \mathsf{QF}(\Sigma)$ where $\mathcal{V}(\psi_{\mathsf{init}}) \cup \mathcal{V}(\psi_{\mathsf{err}}) \subseteq \mathbf{x}$. It is *unsafe* if there are $\mathbf{v}, \mathbf{v}' \in \mathcal{C}^d$ such that $[\mathbf{x}/\mathbf{v}] \models \psi_{\mathsf{init}}$, $\mathbf{v} \rightarrow_\tau^* \mathbf{v}'$, and $[\mathbf{x}/\mathbf{v}'] \models \psi_{\mathsf{err}}$.

Throughout the paper, we use c, d, e, k, ℓ, s for integer constants (where d always denotes the size of $\mathbf{x}$, and s and ℓ always denote the start and length of a loop as in Line 10 of Alg. 1); $\mathbf{v}$ for states; w, x, y for variables; τ, π for relational formulas; σ, θ for valuations; and μ for variable renamings.

3 Transitive Relation Learning

In this section, we present our novel model checking algorithm *Transitive Relation Learning* (TRL) in detail, see Alg. 2. Here, for all $i, j \in \mathbb{N}_+ = \mathbb{N} \setminus \{0\}$ we define $\mu_{i,j}(x') := {}^{(i+j)}x$ if $x' \in \mathbf{x}'$ and $\mu_{i,j}(x) := {}^{(i)}x$, otherwise. So in particular, we have $\mu_{i,j}(\mathbf{x}) = {}^{(i)}\mathbf{x}$ and $\mu_{i,j}(\mathbf{x}') = {}^{(i+j)}\mathbf{x}$, where we assume that ${}^{(1)}\mathbf{x}, {}^{(2)}\mathbf{x}, \ldots \in \mathcal{V}^d$ are disjoint vectors of pairwise different fresh variables. Intuitively, the variables ${}^{(i)}\mathbf{x}$ represent the ith state in a run, and applying $\mu_{i,j}$ to a relational formula yields a formula that relates the ith and the $(i+j)^{th}$ state of a run. For convenience, we define $\mu_i := \mu_{i,1}$ for all $i \in \mathbb{N}_+$, i.e., $\mu_i(\mathbf{x}) = {}^{(i)}\mathbf{x}$ and $\mu_i(\mathbf{x}') = {}^{(i+1)}\mathbf{x}$. As in SMT-based BMC, TRL uses an incremental SMT solver to unroll the transition relation step by step (Line 8), but in contrast to BMC, TRL infers *learned relations* on the fly (Line 16). The *input formula* τ as well as all learned relations are stored in $\boldsymbol{\pi}$. Before each unrolling, we set a backtracking point with the command push and add a suitably variable-renamed version of the description of the error states to the SMT problem (i.e., to the state of the underlying SMT solver) in Line 4. Then the command check_sat checks for reachability of error states, and the command pop removes all formulas from the SMT problem that have been added since the last invocation of push (Line 5), i.e., it removes the encoding of the error states (unless the check succeeds, so that TRL fails). For each unrolling, suitably variable renamed variants of $\boldsymbol{\pi}$'s elements are added to the underlying SMT problem with the command add in Line 8. If no error state is reachable after $b-1$ steps, but the transition relation cannot be unrolled b times (i.e., the SMT problem that corresponds to the b-fold unrolling is unsatisfiable), then the diameter of the analyzed system (including learned relations) has been reached, and hence safety has been proven (Line 10).

The remainder of this section is structured as follows: First, Sect. 3.1 introduces *conjunctive variable projections* that are used to compute the *trace* (Line 11 of Alg. 2). Next, Sect. 3.2 defines *loops* and discusses how to find *non-redundant loops* that are suitable for learning new relations (Lines 12 to 15). Then, Sect. 3.3 introduces *transitive projections* that are used to learn relations (Lines 7 and 16). Afterwards, Sect. 3.4 presents *blocking clauses*, which ensure that learned relations are preferred over other (sequences of) transitions (Lines 9 and 17 to 19). Finally, we illustrate Alg. 2 with a complete example in Sect. 3.5.

3.1 Conjunctive Variable Projections and Traces

To decide when to learn a new relation, TRL inspects the *trace* (Lines 11 and 12). The trace is a sequence of transitions induced by the formulas that have been added to the SMT problem while unrolling the transition relation, and by the current model (Line 11). To compute them, we use *conjunctive variable projections*, which are like *model based projections* [41], but always yield conjunctions.

Definition 2 (Conjunctive Variable Projection)**.** A function

$$\mathsf{cvp} : \mathsf{QF}(\Sigma) \times (\mathcal{V} \rightharpoonup \mathcal{C}) \times 2^{\mathcal{V}} \to \mathsf{QF}(\Sigma)$$

is called a *conjunctive variable projection* if

1. $\sigma \models \mathsf{cvp}(\tau, \sigma, X)$
2. $\mathsf{cvp}(\tau, \sigma, X) \models \tau$
3. $\{\mathsf{cvp}(\tau, \theta, X) \mid \theta \models \tau\}$ is finite
4. $\mathcal{V}(\mathsf{cvp}(\tau, \sigma, X)) \subseteq X \cap \mathcal{V}(\tau)$
5. $\mathsf{cvp}(\tau, \sigma, X) \in \mathsf{QF}_\wedge(\Sigma)$

for all $\tau \in \mathsf{QF}(\Sigma)$, $X \subseteq \mathcal{V}$, and $\sigma \models \tau$. We abbreviate $\mathsf{cvp}(\tau, \sigma, \mathbf{x} \cup \mathbf{x}')$ by $\mathsf{cvp}(\tau, \sigma)$.

So like model based projection, cvp under-approximates quantifier elimination by projecting to the variables X (by (2) and (4)). To do so, it implicitly performs a finite case analysis (by (3)), which is driven by the model σ (by (1)). In contrast to model based projections, cvp always yields conjunctions (by (5)). Note that by (1), $\mathsf{cvp}(\tau, \sigma, X)$ may only contain variables from $\mathrm{dom}(\sigma)$.

Remark 3 (cvp and mbp)**.** Conjunctive variable projections are obtained by combining a model based projection mbp (which satisfies Def. 2 (1–4)) with *syntactic implicant projection* sip [23]. To see how to compute mbp for linear integer arithmetic, recall that Cooper's method for quantifier elimination [14] essentially maps $\exists \mathbf{y}.\ \tau$ to a disjunction of formulas of the form $\tau[\mathbf{y}/\mathbf{t}]$, where the variables $\mathbf{y}$ do not occur in the terms $\mathbf{t}$. Instead, mbp just computes one of these disjuncts, which is satisfied by the provided model. For $\mathsf{sip}(\tau, \sigma)$, one computes the conjunction of all literals of τ's negation normal form that are satisfied by σ. Then $\mathsf{cvp}(\tau, \sigma) := \mathsf{sip}(\mathsf{mbp}(\tau, \sigma), \sigma)$.

Remark 4 (cvp and Quantifier Elimination)**.** Def. 2 (1–4) imply

$$\exists \mathbf{y}.\ \tau \equiv \bigvee_{\sigma \models \tau} \mathsf{cvp}(\tau, \sigma) \qquad \text{where } \mathbf{y} \text{ are } \tau\text{'s extra variables.}$$

So cvp yields a quantifier elimination procedure qe which maps $\exists \mathbf{y}.\ \tau$ to an equivalent quantifier-free formula:

res $\leftarrow \bot$
while τ *has a model* σ
  res $\leftarrow$ *res* $\vee\ \mathsf{cvp}(\tau, \sigma)$
  $\tau \leftarrow \tau \wedge \neg \mathsf{cvp}(\tau, \sigma)$
return *res*

But for a single model σ, $\mathsf{cvp}(\tau, \sigma)$ under-approximates quantifier elimination.

So $\mathsf{cvp}(\tau, \sigma)$ just computes one disjunct of $\mathsf{qe}(\exists \mathbf{y}.\ \tau)$ which is satisfied by σ. However, like model based projection, cvp can be implemented efficiently for many theories with effective, but very expensive quantifier elimination procedures.

Example 5 (cvp)**.** Consider the following formula $\tilde{\tau}$:

$$((w \doteq 0 \wedge \overset{(2)}{x} \doteq x{+}1 \wedge \overset{(2)}{y} \doteq y{+}1) \vee (\overset{(2)}{w} \doteq w \wedge w \doteq 1 \wedge \overset{(2)}{x} \doteq x{-}1 \wedge \overset{(2)}{y} \doteq y{-}1)) \wedge$$
$$((\overset{(2)}{w} \doteq 0 \wedge x' \doteq \overset{(2)}{x}{+}1 \wedge y' \doteq \overset{(2)}{y}{+}1) \vee (w' \doteq \overset{(2)}{w} \wedge \overset{(2)}{w} \doteq 1 \wedge x' \doteq \overset{(2)}{x}{-}1 \wedge y' \doteq \overset{(2)}{y}{-}1))$$

It encodes two steps with Ex. 1, where $\overset{(2)}{\mathbf{x}} = [\overset{(2)}{w}, \overset{(2)}{x}, \overset{(2)}{y}]$ represents the values after one step. In Line 11, Alg. 2 might find a run like $\sigma(\overset{(1)}{\mathbf{x}}) \to_\tau \sigma(\overset{(2)}{\mathbf{x}}) \to_\tau \sigma(\overset{(3)}{\mathbf{x}})$ for

$$\sigma := [\overset{(1)}{w}/\overset{(1)}{x}/\overset{(1)}{y}/0,\ \overset{(2)}{w}/\overset{(2)}{x}/\overset{(2)}{y}/1,\ \overset{(3)}{w}/1, \overset{(3)}{x}/\overset{(3)}{y}/0].$$

Here, $[w/x/y/c, \ldots]$ abbreviates $[w/c, x/c, y/c, \ldots]$. Then the variable renaming $\mu_{1,2}$ allows us to instantiate the pre- and post-variables by the first and last state, resulting in the following model of $\tilde{\tau}$:

$$\tilde{\sigma} := \sigma \circ \mu_{1,2} = \sigma \cup [w/x/y/0,\ \ w'/1, x'/y'/0] \quad \text{where } (\sigma \circ \mu_{1,2})(x) = \sigma(\mu_{1,2}(x))$$

To get rid of $\overset{(2)}{w}, \overset{(2)}{x}, \overset{(2)}{y}$, one could compute $\mathsf{qe}(\exists \overset{(2)}{w}, \overset{(2)}{x}, \overset{(2)}{y}.\ \tilde{\tau})$, resulting in:

$$w \doteq 0 \wedge x' \doteq x + 2 \wedge y' \doteq y + 2 \qquad (\mathsf{inc})$$
$$\vee\ w \doteq 0 \wedge w' \doteq 1 \wedge x' \doteq x \wedge y' \doteq y \qquad (\mathsf{eq})$$
$$\vee\ w \doteq 1 \wedge w' \doteq 1 \wedge x' \doteq x - 2 \wedge y' \doteq y - 2. \qquad (\mathsf{dec})$$

Instead, we may have $\mathsf{cvp}(\tilde{\tau}, \tilde{\sigma}) = (\mathsf{eq})$, as $\tilde{\sigma} \models (\mathsf{eq})$.

Intuitively, a relational formula τ describes how states can change, so it is composed of many different cases. These cases may be given explicitly (by disjunctions) or implicitly (by extra variables, which express non-determinism). Given a model σ of τ that describes a *concrete* change of state, cvp computes a description of the corresponding case. Computing *all* cases amounts to eliminating all extra variables and converting the result to DNF, which is impractical.

When unrolling the transition relation in Line 8 of Alg. 2, we identify each relational formula π_n with its index n in the sequence $\boldsymbol{\pi}$. To this end, we use a fresh variable x_{id}, and our SMT encoding forces $\overset{(i)}{x}_{\mathsf{id}}$ to be the identifier of the relation that is used for the ith step. Similarly to [26], the *trace* is the sequence of transitions that results from applying cvp to the unrolling of the transition relation that is constructed by Alg. 2 in Line 8. So a trace is a sequence of transitions that can be applied subsequently, starting in an initial state.

Definition 6 (Trace)**.** Let $\boldsymbol{\pi}$ be a sequence of relational formulas, let

$$\sigma \models \bigwedge_{i=1}^{b} \mu_i \left(\bigvee_{n=1}^{|\boldsymbol{\pi}|} (\pi_n \wedge x_{\mathsf{id}} \doteq n) \right) \qquad \text{where } b \in \mathbb{N}_+, \tag{3}$$

and let $\mathsf{id}(i) := \sigma(\overset{(i)}{x}_{\mathsf{id}})$. Then the *trace induced by* σ is

$$\mathsf{trace}_b(\sigma, \boldsymbol{\pi}) := [\mathsf{cvp}(\pi_{\mathsf{id}(i)}, \sigma \circ \mu_i)]_{i=1}^{b}.$$

Recall that μ_i renames $\mathbf{x}$ and $\mathbf{x}'$ into $\overset{(i)}{\mathbf{x}}$ and $\overset{(i+1)}{\mathbf{x}}$, and $\mathsf{id}(i) = \sigma(\overset{(i)}{x}_{\mathsf{id}})$ is the index of the relation from $\boldsymbol{\pi}$ that is used for the ith step. So each model σ of (3) corresponds to a run

$$\sigma(\mu_1(\mathbf{x})) \to_{\pi_{\mathsf{id}(1)}} \ldots \to_{\pi_{\mathsf{id}(b-1)}} \sigma(\mu_b(\mathbf{x})) \to_{\pi_{\mathsf{id}(b)}} \sigma(\mu_b(\mathbf{x}')),$$

and the trace induced by σ contains the transitions that were used in this run.

Example 7 (Trace)**.** Consider the extension of σ from Ex. 5 with $[\overset{(1)}{x}_{\mathsf{id}}/1,\ \overset{(2)}{x}_{\mathsf{id}}/1]$:

$$\sigma := [\overset{(1)}{w}/\overset{(1)}{x}/\overset{(1)}{y}/0, \overset{(1)}{x}_{\mathsf{id}}/1, \quad \overset{(2)}{w}/\overset{(2)}{x}/\overset{(2)}{y}/\overset{(2)}{x}_{\mathsf{id}}/1, \quad \overset{(3)}{w}/1, \overset{(3)}{x}/\overset{(3)}{y}/0]$$

Thus, $\mathsf{id}(1) = \sigma(\overset{(1)}{x}_{\mathsf{id}}) = 1$, $\mathsf{id}(2) = \sigma(\overset{(2)}{x}_{\mathsf{id}}) = 1$, and $\pi_{\mathsf{id}(1)} = \pi_{\mathsf{id}(2)} = \pi_1 = \tau$. Then

$$\begin{aligned}
&\mathsf{trace}_2(\sigma, [\tau, \tau]) \\
&= [\mathsf{cvp}(\tau, \sigma \circ \mu_1), \mathsf{cvp}(\tau, \sigma \circ \mu_2)] \\
&= [\mathsf{cvp}(\tau, [w/x/y/0,\ w'/x'/y'/1]),\ \mathsf{cvp}(\tau, [w/x/y/1,\ w'/1, x'/y'/0])] \\
&= [\tau_{\mathsf{inc}}, \tau_{\mathsf{dec}}].
\end{aligned}$$

3.2 Loops

As $\boldsymbol{\pi}$ only gives rise to finitely many transitions, the trace is bound to contain *loops*, eventually (unless Alg. 2 terminates beforehand).

Definition 8 (Loop)**.** A sequence of transitions $\tau_1, \ldots, \tau_k$ is called a *loop* if there are $\mathbf{v}_0, \ldots, \mathbf{v}_{k+1} \in \mathcal{C}^d$ such that $\mathbf{v}_0 \to_{\tau_1} \ldots \to_{\tau_k} \mathbf{v}_k \to_{\tau_1} \mathbf{v}_{k+1}$.

Intuitively, these loops are the reason why BMC may diverge. To prevent divergence, TRL learns a new relation when a loop is detected (Line 12).

Remark 9 (Finding Loops)**.** Loops can be detected by SMT solving. A cheaper way is to look for duplicates on the trace, but then loops are found "later", as a trace $[\ldots, \pi, \pi, \ldots]$ is needed to detect a loop π, but one occurrence of π is insufficient. As a trade-off between precision and efficiency, our implementation uses an approximation based on *dependency graphs* [26]. More precisely, our implementation maintains a graph $\mathcal{G}$ whose nodes are transitions, and it adds an edge between two transitions if they occur subsequently on the trace at some point. Then it considers a subsequence $\tau_i, \ldots, \tau_j$ of the trace to be a potential loop if $\mathcal{G}$ contains an edge from τ_j to τ_i.

Remark 10 (Disregarding "Learned" Loops)**.** One should disregard "loops" consisting of a single *learned transition*, i.e., a transition that results from applying cvp to some $\pi \in \mathsf{tail}(\boldsymbol{\pi})$, where $\mathsf{tail}(\tau :: \boldsymbol{\pi}') := \boldsymbol{\pi}'$. Here, $\mathsf{tail}(\boldsymbol{\pi})$ contains all learned relations, as

the first element of $\boldsymbol{\pi}$ is the input formula τ. The reason is that our goal is to deduce transitive relations, but learned relations are already transitive. In the sequel, we assume that the check in Line 12 fails for such loops.

If there are several choices for s and ℓ in Line 12, then our implementation only considers loops of minimal length and, among those, it minimizes s.

Example 11 (Detecting Loops)**.** Consider the following model for τ from Ex. 1.

$$\sigma := [\overset{(1)}{w}/\overset{(1)}{x}/\overset{(1)}{y}/0, \overset{(1)}{x}_{\mathsf{id}}/1, \quad \overset{(2)}{w}/0, \overset{(2)}{x}/\overset{(2)}{y}/1]$$

Then $\mathsf{trace}_1(\sigma, [\tau]) = [\tau_{\mathsf{inc}}]$. As τ_{inc} is a loop,[4] this causes TRL to learn a relation like the following one (see Sect. 3.3 for details).

$$w \doteq 0 \wedge x' > x \wedge x' - x \doteq y' - y \qquad (\tau^+_{\mathsf{inc}})$$

TRL only learns relations from loops that are *non-redundant* w.r.t. all relations that have been learned before [23].

Definition 12 (Redundancy)**.** If $\rightarrow_\tau \subseteq \rightarrow_{\tau'}$, then τ is *redundant* w.r.t. τ'.

Example 13. The relation τ_{inc} is redundant w.r.t. τ^+_{inc}, but τ_{dec} is not.

Line 14 uses a sufficient criterion for non-redundancy: If all learned relations are falsified by the values before and after the loop, then $\tau_s, \ldots, \tau_{s+\ell-1}$ cannot be simulated by a previously learned relation, so it is non-redundant and we learn a new relation. The values before and after the loop are obtained from the current model σ by setting $\mathbf{x}$ to $\sigma(\overset{(s)}{\mathbf{x}})$ and $\mathbf{x}'$ to $\sigma(\overset{(s+\ell)}{\mathbf{x}})$, i.e., we use $\sigma \circ \mu_{s,\ell}$ in Line 13.

To learn a new relation, we first compute the relation

$$\tau_{\mathsf{loop}} := \mu^{-1}_{s,\ell}(\varphi_{\mathsf{loop}}) \qquad \text{where} \qquad \varphi_{\mathsf{loop}} := \bigwedge_{i=s}^{s+\ell-1} \mu_i(\tau_i)$$

of the loop in Line 15, where $\mu^{-1}_{s,\ell}$ is the inverse of $\mu_{s,\ell}$. So in Ex. 11, we have $\sigma \circ \mu_{1,1} \supseteq [w/x/y/0, w'/0, x'/y'/1]$ and $\tau_{\mathsf{loop}} := \mu^{-1}_{1,1}(\mu_1(\tau_{\mathsf{inc}})) = \tau_{\mathsf{inc}}$ as $s = \ell = 1$. So $\sigma \circ \mu_{s,\ell}$ indeed corresponds to one evaluation of the loop, as $\sigma \circ \mu_{s,\ell} \models \tau_{\mathsf{loop}}$.

To see that τ_{loop} is also the desired relation in general, note that φ_{loop} is the conjunction of the transitions that constitute the loop, where all variables are renamed as in Line 8 of Alg. 2, i.e., in such a way that the post-variables of the ith step are equal to the pre-variables of the $(i+1)^{th}$ step. So we have $\sigma \models \varphi_{\mathsf{loop}}$ and thus $\sigma \circ \mu_{s,\ell} \models \tau_{\mathsf{loop}}$. Hence, we can use τ_{loop} and $\sigma \circ \mu_{s,\ell}$ to learn a new relation via *transitive projections* in Line 16.

[4] Depending on the technique that is used to detect loops, an actual implementation might require one more unrolling of $\rightarrow_\tau$ to obtain the trace $[\tau_{\mathsf{inc}}, \tau_{\mathsf{inc}}]$ in order to detect the loop τ_{inc}, see Remark 9.

3.3 Transitive Projections

We now define *transitive projections* that approximate transitive closures of loops. As explained in Sect. 1, we do not restrict ourselves to under- or over-approximations, but we allow "mixtures" of both. Analogously to cvp, transitive projections perform a finite case analysis that is driven by the provided model σ.

Definition 14 (Transitive Projection)**.** A function

$$\mathsf{tp} : \mathsf{QF}(\Sigma) \times (\mathcal{V} \rightharpoonup \mathcal{C}) \to \mathsf{QF}(\Sigma)$$

is called a *transitive projection* if the following holds for all transitions $\tau \in \mathsf{QF}(\Sigma)$ and all $\sigma \models \tau$:

1. $\mathsf{tp}(\tau, \sigma)$ is consistent with σ
2. $\{\mathsf{tp}(\tau, \theta) \mid \theta \models \tau\}$ is finite
3. $\to_{\mathsf{tp}(\tau,\sigma)}$ is transitive

Clearly, the specifics of tp depend on the underlying theory. Our implementation of tp for quantifier-free linear integer arithmetic is explained in Sect. 4.

Example 15 (tp)**.** For Ex. 1, $\tau_{\mathsf{ti}} := x' - x \doteq y' - y$ over-approximates the transitive closure $\to_\tau^+$. Such over-approximations are also called *transition invariants* [53]. With τ_{ti}, one can prove safety for any ψ_{init} with $\psi_{\mathsf{init}} \models x \doteq y$, as then

$$\psi_{\mathsf{init}} \wedge \tau_{\mathsf{ti}} \models x' \doteq y',$$

which shows that no error state with $w \doteq 1 \wedge x \leq 0 \wedge y > 0$ is reachable.

By using cvp, TRL instead considers τ_{inc} and τ_{dec} separately and learns

$$\mathsf{tp}(\tau_{\mathsf{inc}}, \sigma_{\mathsf{inc}}) := w \doteq 0 \wedge x' > x \wedge x' - x \doteq y' - y \qquad (\tau_{\mathsf{inc}}^+)$$
$$\mathsf{tp}(\tau_{\mathsf{dec}}, \sigma_{\mathsf{dec}}) := w' \doteq w \wedge w \doteq 1 \wedge x' < x \wedge x' - x \doteq y' - y \qquad (\tau_{\mathsf{dec}}^+)$$

if $\sigma_{\mathsf{inc}} \models \tau_{\mathsf{inc}}$ and $\sigma_{\mathsf{dec}} \models \tau_{\mathsf{dec}}$. In this way, Alg. 2 can learn disjunctive relations like $\tau_{\mathsf{inc}}^+ \vee \tau_{\mathsf{dec}}^+$, even if tp only yields conjunctive relational formulas (which is true for our current implementation of tp – see Sect. 4 – but not enforced by Def. 14).

In contrast to conjunctive variable projections, $\mathsf{tp}(\tau, \sigma)$ may contain extra variables that do not occur in τ (which will be exploited in Sect. 4). Hence, instead of $\sigma \models \mathsf{tp}(\tau, \sigma)$ we require consistency with σ, i.e., $\sigma(\mathsf{tp}(\tau, \sigma))$ must be satisfiable.

Remark 16 (Properties of tp)**.** Due to Def. 14 (1), our definition of tp implies

$$\tau \models \exists \mathbf{y}. \bigvee_{\sigma \models \tau} \mathsf{tp}(\tau, \sigma), \quad \text{and thus,} \quad \to_\tau \subseteq \bigcup_{\sigma \models \tau} \to_{\mathsf{tp}(\tau,\sigma)},$$

where $\mathbf{y}$ are the extra variables of $\bigvee_{\sigma \models \tau} \mathsf{tp}(\tau, \sigma)$. However, Def. 14 does *not* ensure $\to_\tau^+ \subseteq \bigcup_{\sigma \models \tau} \to_{\mathsf{tp}(\tau,\sigma)}$. So there is no guarantee that tp covers $\to_\tau^+$ entirely, i.e., tp cannot be used to compute transition invariants, in general. Def. 14 does not ensure $\to_\tau^+ \supseteq \bigcup_{\sigma \models \tau} \to_{\mathsf{tp}(\tau,\sigma)}$ either, as $\mathsf{tp}(\tau, \sigma)$ does not imply $\sigma(\mathbf{x}) \to_\tau^+ \sigma(\mathbf{x}')$.

Example 17. To see that tp computes no over- or under-approximations, let

$$\tau := x' \doteq x + 1 \wedge y' \doteq y + x.$$

Then for all $\sigma \models \tau$, we might have:

$$\mathsf{tp}(\tau, \sigma) = \begin{cases} x \geq 0 \wedge x' > x \wedge y' \geq y, & \text{if } \sigma(x) \geq 0 \\ x < 0 \wedge x' > x \wedge y' < y, & \text{if } \sigma(x) < 0 \end{cases}$$

However,

$$(x \geq 0 \wedge x' > x \wedge y' \geq y) \vee (x < 0 \wedge x' > x \wedge y' < y)$$

is not an over-approximation of $\to_\tau^+$ (i.e., $\to_\tau^+ \not\subseteq \bigcup_{\sigma \models \tau} \to_{\mathsf{tp}(\tau,\sigma)}$), as we have, e.g.,

$$(-1, 0) \to_\tau (0, -1) \to_\tau (1, -1) \to_\tau (2, 0), \qquad \text{but} \qquad (-1, 0) \not\to_{\mathsf{tp}(\tau,\sigma)} (2, 0)$$

for all $\sigma \models \tau$. Moreover, we also have $\to_\tau^+ \not\supseteq \bigcup_{\sigma \models \tau} \to_{\mathsf{tp}(\tau,\sigma)}$, since

$$(-1, 0) \to_{\mathsf{tp}(\tau,\sigma)} (10, -20), \qquad \text{but} \qquad (-1, 0) \not\to_\tau^+ (10, -20)$$

if $\sigma(x) < 0$. In contrast to $\mathsf{tp}(\tau, \sigma)$, linear over-approximations for $\to_\tau^+$ like $x' > x$ cannot distinguish whether y increases or decreases.

As TRL proves safety via *blocking clauses* (Sect. 3.4) that only block steps that are covered by learned relations, the fact that tp does not yield over-approximations does not affect soundness. However, it may cause divergence (Remark 23).

Recall that our SMT encoding forces $\overset{(i)}{x}_{\mathsf{id}}$ to be the identifier of the relation that is used for the ith step (Line 8). To exploit transitivity of tp, we add the constraint $\overset{(b)}{x}_{\mathsf{id}} \doteq 1 \vee \overset{(b)}{x}_{\mathsf{id}} \neq \overset{(b-1)}{x}_{\mathsf{id}}$ in Line 7, so that learned relations (with index > 1) are not used several times in a row. The reason is that using transitive relations twice in a row is unnecessary, as all states that are reachable in two steps can also be reached in one step.

3.4 Blocking Clauses

In Line 17, we are guaranteed to find a learned relation π which is consistent with σ_{loop}: If our sufficient criterion for non-redundancy in Line 14 failed, then the existence of π is guaranteed. Otherwise, we learned a new relation π in Line 16 which is consistent with $\sigma_{\mathsf{loop}} \subseteq \sigma \circ \mu_{s,\ell}$ by definition of tp. Thus, we can use π and a model $\overline{\sigma} \supseteq \sigma_{\mathsf{loop}}$ of π to record a *blocking clause* in Line 18.

Definition 18 (Blocking Clauses)**.** Consider a relational formula π, and let $\overline{\sigma}$ be a model of π. We define:

$$\mathsf{blocking_clause}(s, \ell, \pi, \overline{\sigma}) := \begin{cases} \mu_{s,\ell}(\neg\mathsf{cvp}(\pi, \overline{\sigma})) \vee \overset{(s)}{x}_{\mathsf{id}} > 1, & \text{if } \ell = 1 \\ \mu_{s,\ell}(\neg\mathsf{cvp}(\pi, \overline{\sigma})), & \text{if } \ell > 1 \end{cases}$$

Here, s and ℓ are natural numbers such that $[\tau_i]_{i=s}^{s+\ell-1}$ is a (possibly) redundant loop on the trace. Blocking clauses exclude models that correspond to runs

$$\mathbf{v}_1 \to_{\tau_1} \ldots \to_{\tau_{s-1}} \mathbf{v}_s \to_{\tau_s} \ldots \to_{\tau_{s+\ell-1}} \mathbf{v}_{s+\ell} \tag{4}$$

where $\mathbf{v}_s \to_\pi \mathbf{v}_{s+\ell}$. Intuitively, if $\ell = 1$ then $\mathsf{blocking_clause}(s, \ell, \pi, \overline{\sigma})$ states that one may still evaluate $\mathbf{v}_s$ to $\mathbf{v}_{s+\ell}$, but one has to use a learned transition. If $\ell > 1$, then $\mathsf{blocking_clause}(s, \ell, \pi, \overline{\sigma})$ states that one may still evaluate $\mathbf{v}_s$ to $\mathbf{v}_{s+\ell}$, but not in ℓ steps. More precisely, blocking clauses take into account that

$$\mathbf{v}_1 \to_{\tau_1} \ldots \to_{\tau_{s+\ell-2}} \mathbf{v}_{s+\ell-1} \tag{5}$$

and

$$\mathbf{v}_1 \to_{\tau_1} \ldots \to_{\tau_{s-1}} \mathbf{v}_s \to_\pi \mathbf{v}_{s+\ell} \tag{6}$$

must not be blocked to ensure that $\mathbf{v}_2, \ldots, \mathbf{v}_{s+\ell}$ remain reachable. For the former, note that blocking clauses affect the suffix $\mathbf{v}_s \to_{\tau_s} \ldots \to_{\tau_{s+\ell-1}} \mathbf{v}_{s+\ell}$ of (4) (as they contain $\mu_{s,\ell}(\neg\mathsf{cvp}(\pi, \overline{\sigma}))$), but not (5), so $\mathbf{v}_2, \ldots, \mathbf{v}_{s+\ell-1}$ remain reachable.

Regarding (6), note that (4) corresponds to one unrolling of the loop (without using the newly learned relation π). In contrast, (6) simulates one unrolling of the loop using the new relation π. To see that our blocking clauses prevent the sequence (4), but not the sequence (6), first consider the case $\ell > 1$. Then (6) is not affected by the blocking clause, as it requires less than $s + \ell$ steps. If $\ell = 1$, then the loop that needs to be blocked is a single *original transition* (i.e., a transition that results from applying cvp to τ) due to Remark 10. So $\overset{(s)}{x}_{\mathsf{id}} > 1$ is falsified by (4), as τ_s is an original transition, i.e., using it for the sth step implies $\overset{(s)}{x}_{\mathsf{id}} \doteq 1$. However, $\overset{(s)}{x}_{\mathsf{id}} > 1$ is satisfied by (6), as π is a learned transition, so using it implies $\overset{(s)}{x}_{\mathsf{id}} > 1$.

Remark 19 (Extra Variables and Negation)**.** In Def. 18, cvp is used to project π according to the model $\overline{\sigma}$. In this way, negation has the intended effect, i.e., we have

$$[\mathbf{x}/\mathbf{v}, \mathbf{x}'/\mathbf{v}'] \models \neg\mathsf{cvp}(\ldots) \qquad \text{iff} \qquad \mathbf{v} \not\to_{\mathsf{cvp}(\ldots)} \mathbf{v}',$$

as $\mathsf{cvp}(\ldots)$ has no extra variables. To see why this is important here, consider the relation $\tau := n > 0 \wedge x' \doteq x + n$, where n is an extra variable. Then $0 \to_\tau 1$, but

$$\neg\tau[x/0, x'/1] = (n \leq 0 \vee x' \not\doteq x + n)[x/0, x'/1] = n \leq 0 \vee 1 \not\doteq n$$

is satisfiable, so $\neg\tau$ is not a suitable characterization of $\not\to_\tau$. The reason is that n is implicitly existentially quantified in τ. So to characterize $\not\to_\tau$, we have to negate $\exists n.\ \tau$ instead of τ, resulting in $\forall n.\ n \leq 0 \vee x' \neq x + n$. Then, as desired,

$$(\forall n.\ n \leq 0 \vee x' \neq x + n)[x/0, x'/1] = \forall n.\ n \leq 0 \vee 1 \neq n$$

is invalid. To avoid quantifiers, we eliminate extra variables via cvp instead.

In Line 18, a pair consisting of $s + \ell - 1$ and the blocking clause is added to BLOCKED. The first component means that the blocking clause has to be added to the SMT encoding when the transition relation is unrolled for the $(s + \ell - 1)^{th}$ time, i.e., when $b = s + \ell - 1$. So blocking clauses are added to the SMT encoding "on demand" (in Line 9) to block loops that have been found on the trace at some point. Afterwards, TRL backtracks to the last step before the loop in Line 19.

Remark 20 (Adding Blocking Clauses)**.** To see why blocking clauses must only be added to the SMT encoding in the $(s + \ell - 1)^{th}$ unrolling, assume $\pi \equiv \top$. Then, e.g., $\mathsf{blocking_clause}(1, 2, \pi, \overline{\sigma}) \equiv \bot$. This means that unrolling the transition relation twice is superfluous, as every state can be reached in a single step with π, so the diameter is 1. But after learning π when $b = 2$ and backtracking to $b = 0$, adding such a blocking clause too early (e.g., before the first unrolling of the transition relation) would *immediately* result in an unsatisfiable SMT problem.

Example 21 (Blocking Redundant Loops)**.** Consider the model

$$\sigma := [\overset{(1)}{w}/\overset{(1)}{x}/\overset{(1)}{y}/0, \overset{(1)}{x}_{\mathsf{id}}/2, \quad \overset{(2)}{w}/0, \overset{(2)}{x}/\overset{(2)}{y}/2, \overset{(2)}{x}_{\mathsf{id}}/1, \quad \overset{(3)}{x}/\overset{(3)}{y}/3]$$

and assume that TRL has already learned the relation τ^+_{inc} (i.e., $\boldsymbol{\pi} = [\tau, \tau^+_{\mathsf{inc}}]$). Moreover, assume that the trace is $[\tau^+_{\mathsf{inc}}, \tau_{\mathsf{inc}}]$, so that TRL detects the loop τ_{inc}. To check for non-redundancy, we instantiate the pre- and post-variables in τ^+_{inc} according to σ, taking the renaming μ_2 into account (note that here $s = 2$, $\ell = 1$, and $\mu_{s,\ell} = \mu_{2,1} = \mu_2$):

$$\sigma(\mu_2(\tau^+_{\mathsf{inc}})) = \tau^+_{\mathsf{inc}}[w/0, x/y/2, x'/y'/3] \equiv \top.$$

So our sufficient criterion for non-redundancy fails, as τ_{inc} is indeed redundant w.r.t. τ^+_{inc}. Thus, TRL records that the following blocking clause has to be added for the second unrolling (i.e., when $b = s + \ell - 1 = 2$).

$$\begin{aligned}
&\mu_{s,\ell}(\neg\mathsf{cvp}(\tau^+_{\mathsf{inc}}, \overline{\sigma})) \vee \overset{(s)}{x}_{\mathsf{id}} > 1 \\
&= \mu_{s,\ell}(\neg\tau^+_{\mathsf{inc}}) \vee \overset{(s)}{x}_{\mathsf{id}} > 1 && \text{(as } \tau^+_{\mathsf{inc}} \text{ is a transition)} \\
&= \mu_2(\neg(w \doteq 0 \wedge x' > x \wedge x' - x \doteq y' - y)) \vee \overset{(2)}{x}_{\mathsf{id}} > 1 \\
&\equiv (w \neq 0 \vee x' \leq x \vee x' - x \neq y' - y)[w/\overset{(2)}{w}, x/\overset{(2)}{x}, y/\overset{(2)}{y}, x'/\overset{(3)}{x}, y'/\overset{(3)}{y}] \vee \overset{(2)}{x}_{\mathsf{id}} > 1 \\
&= \overset{(2)}{w} \neq 0 \vee \overset{(3)}{x} \leq \overset{(2)}{x} \vee \overset{(3)}{x} - \overset{(2)}{x} \neq \overset{(3)}{y} - \overset{(2)}{y} \vee \overset{(2)}{x}_{\mathsf{id}} > 1
\end{aligned}$$

As this blocking clause is falsified by σ, it prevents TRL from finding the same model again after backtracking in Line 19, so that TRL makes progress.

The following theorem states that our approach is sound.

Theorem 22. *If Alg. 2 returns* safe*, then $\mathcal{T}$ is safe.*

Proof (Sketch) The key idea of the proof, which works by induction over the length of runs, is to show that blocking clauses do not prevent us from reaching all reachable states. See App. A for the full proof. □

Remark 23 (Termination)**.** In general, Alg. 2 does not terminate, since tp decomposes the relation into finitely many cases and approximates their transitive closures independently, but $\to^+_{\tau_{\mathsf{loop}}} \subseteq \bigcup_{\sigma \models \tau_{\mathsf{loop}}} \to_{\mathsf{tp}(\tau_{\mathsf{loop}},\sigma)}$ is not guaranteed (Remark 16). To see why this may prevent termination, consider a loop τ_{loop} and assume that there are reachable states $\mathbf{v}, \mathbf{v}'$ with $\mathbf{v} \to^+_{\tau_{\mathsf{loop}}} \mathbf{v}'$, but $\mathbf{v} \not\to_{\mathsf{tp}(\tau_{\mathsf{loop}},\sigma)} \mathbf{v}'$ for all models σ of τ_{loop}. Then TRL may find a model that corresponds to a run from $\mathbf{v}$ to $\mathbf{v}'$. Unless $\mathbf{v}$ can be evaluated to $\mathbf{v}'$ with another learned transition $\pi \notin \{\mathsf{tp}(\tau_{\mathsf{loop}}, \sigma) \mid \sigma \models \tau_{\mathsf{loop}}\}$ by coincidence, this loop cannot be blocked and TRL learns a new relation. Thus, TRL may keep learning new relations as long as there are loops whose transitive closure is not yet covered by learned relations.

As the elements of $\{\mathsf{tp}(\tau, \sigma) \mid \sigma \models \tau\}$ are independent of each other, a more "global" view may help to enforce convergence. We leave that to future work.

3.5 A Complete Example

For a complete run of TRL on our example, assume that we obtain the following traces (where the detected loops are underlined):

1. $[\underline{\tau_{\mathsf{inc}}}]$, resulting in the learned relation τ^+_{inc} and the blocking clause $\mu_1(\neg\tau^+_{\mathsf{inc}} \vee x_{\mathsf{id}} > 1)$ which ensures that if $b = 1$, then we cannot use τ_{inc} but would have to use τ^+_{inc}.
2. $[\underline{\tau_{\mathsf{dec}}}]$, resulting in the learned relation τ^+_{dec} and the blocking clause $\mu_1(\neg\tau^+_{\mathsf{dec}} \vee x_{\mathsf{id}} > 1)$ which ensures that if $b = 1$, then we cannot use τ_{dec} but would have to use τ^+_{dec}.
3. $[\tau^+_{\mathsf{inc}}, \underline{\tau_{\mathsf{inc}}}]$, resulting in the blocking clause $\mu_2(\neg\tau^+_{\mathsf{inc}} \vee x_{\mathsf{id}} > 1)$ which ensures that if $b = 2$, then we cannot use τ_{inc}. Using τ^+_{inc} twice after each other is also not possible due to transitivity (Line 7).
4. $[\tau^+_{\mathsf{dec}}, \underline{\tau_{\mathsf{dec}}}]$, resulting in the blocking clause $\mu_2(\neg\tau^+_{\mathsf{dec}} \vee x_{\mathsf{id}} > 1)$ which ensures that if $b = 2$, then we cannot use τ_{dec}. Using τ^+_{dec} twice after each other is also not possible due to transitivity (Line 7).
5. $[\tau^+_{\mathsf{inc}}, \tau^+_{\mathsf{dec}}, \underline{\tau_{\mathsf{dec}}}]$, resulting in the blocking clause $\mu_3(\neg\tau^+_{\mathsf{dec}} \vee x_{\mathsf{id}} > 1)$ which ensures that if $b = 3$, then we cannot use τ_{dec}.

Now we are in the following situation:

- The first element of the trace cannot be τ_{inc} or τ_{dec} due to (1) and (2).

- If the first element of the trace is τ^+_{inc}, then:
  - The second element of the trace cannot be τ_{inc} or τ_{dec} due to (3) and (4).
  - The second element of the trace cannot be τ^+_{inc} due to Line 7.
  - If the second element of the trace is τ^+_{dec}, then:
    * The third element of the trace cannot be τ_{inc} or τ^+_{inc}, as τ^+_{dec} sets w to 1, but τ_{inc} and τ^+_{inc} require $w = 0$.
    * The third element of the trace cannot be τ_{dec} due to (5).
    * The third element of the trace cannot be τ^+_{dec} due to Line 7.

  So this case becomes infeasible with the 3rd unrolling of the transition relation.
- If the first element of the trace is τ^+_{dec}, then:
  - The second element of the trace cannot be τ_{inc} or τ^+_{inc}, as τ^+_{dec} sets w to 1, but τ_{inc} and τ^+_{inc} require $w = 0$.
  - The second element of the trace cannot be τ_{dec} due to (4).
  - The second element of the trace cannot be τ^+_{dec} due to Line 7.

  So this case becomes infeasible with the 2nd unrolling of the transition relation.

Thus, the underlying SMT problem becomes unsatisfiable when $b = 3$, such that safe is returned in Line 10.

4 Implementing tp for Linear Integer Arithmetic

We now explain how to compute transitive projections for quantifier-free linear integer arithmetic via recurrence analysis. As in SMT-LIB [4], in our setting linear integer arithmetic also features (in)divisibility predicates of the form $e|t$ (or $e\not|t$) where $e \in \mathbb{N}_+$ and t is an integer-valued term. Then we have $\sigma \models e|t$ iff $\sigma(t)$ is a multiple of e, and $\sigma \models e\not|t$, otherwise.

The technique that we use is inspired by the recurrence analysis from [20]. However, there are some important differences. The approach from [20] computes convex hulls to over-approximate disjunctions by conjunctions, and it relies on polyhedral projections. In our setting, we always have a suitable model at hand, so that we can use cvp instead. Hence, our recurrence analysis can be implemented more efficiently.[5] Additionally, our recurrence analysis can handle divisibility predicates, which are not covered in [20].

On the other hand, [20] yields an over-approximation of the transitive closure of the given relation, whereas our approach performs an implicit case analysis (via cvp) and only yields an over-approximation of the transitive closure of one out of finitely many cases.

Moreover, the recurrence analysis from [20] also discovers non-linear relations, and then uses linearization techniques to eliminate them. For simplicity, our recurrence analysis only derives linear relations so far. However, just like [20], we could also derive non-linear relations and linearize them afterwards. Apart from these differences, our technique is analogous to [20].

[5] The *double description method*, which is popular for computing polyhedral projections and convex hulls, and other state-of-the-art approaches have exponential complexity [9, 55]. See [31] for an easily accessible discussion of the complexity of the double description method. In contrast, combining the model based projection from [41] with syntactic implicant projection [23] yields a polynomial time algorithm for cvp.

In the sequel, let τ and $\sigma \models \tau$ be fixed. Our implementation of $\mathsf{tp}(\tau, \sigma)$ first searches for *recurrent literals*, i.e., literals of the form[6]

$$t \bowtie 0 \text{ or } e|t \quad \text{where} \quad t = \sum_{x \in \mathbf{x}} c_x \cdot (x' - x) + c,\ \bowtie \in \{\leq, \geq, <, >, \doteq\},\ \text{and } c_x, c \in \mathbb{Z}.$$

Hence, these literals provide information about the change of values of variables. To find such literals, we introduce a fresh variable x_δ for each $x \in \mathbf{x}$, and we conjoin $x_\delta \doteq x' - x$ to τ, i.e., we compute

$$\tau_{\wedge\delta} := \tau \wedge \bigwedge_{x \in \mathbf{x}} x_\delta \doteq x' - x.$$

So the value of x_δ corresponds to the change of x when applying τ. Next, we use cvp to eliminate all variables but $\{x_\delta \mid x \in \mathbf{x}\}$ from $\tau_{\wedge\delta}$, resulting in τ_δ. More precisely, we have

$$\tau_\delta := \mathsf{cvp}(\tau_{\wedge\delta}, \quad \sigma \uplus [x_\delta / \sigma(x' - x) \mid x \in \mathbf{x}], \quad \{x_\delta \mid x \in \mathbf{x}\}).$$

Finally, to obtain a formula where all literals are recurrent, we replace each x_δ by its definition, i.e., we compute

$$\tau_{\mathsf{rec}} := \tau_\delta[x_\delta / x' - x \mid x \in \mathbf{x}].$$

Example 24 (Finding Recurrent Literals)**.** Consider the transition τ_{dec}. We first construct the formula

$$\tau_{\wedge\delta} := \tau_{\mathsf{dec}} \wedge w_\delta \doteq w' - w \wedge x_\delta \doteq x' - x \wedge y_\delta \doteq y' - y.$$

Then for any model $\sigma \models \tau_{\mathsf{dec}}$, we get[7]

$$\begin{aligned} \tau_\delta &:= \mathsf{cvp}(\tau_{\wedge\delta}, \quad \sigma \uplus [w_\delta / 0,\ x_\delta / -1,\ y_\delta / -1], \quad \{w_\delta, x_\delta, y_\delta\}) \\ &= w_\delta \doteq 0 \wedge x_\delta \doteq -1 \wedge y_\delta \doteq -1. \end{aligned}$$

Next, replacing w_δ, x_δ, and y_δ with their definition results in

$$\begin{aligned} \tau_{\mathsf{rec}} &:= w' - w \doteq 0 \wedge x' - x \doteq -1 \wedge y' - y \doteq -1 \\ &\equiv w' - w \doteq 0 \wedge x' - x + 1 \doteq 0 \wedge y' - y + 1 \doteq 0. \end{aligned}$$

Then the construction of $\mathsf{tp}(\tau, \sigma)$ proceeds as follows:

[6] W.l.o.g., we assume that literals are never negated, as we can negate the corresponding (in)equalities or divisibility predicates directly instead. Furthermore, in our implementation, we replace disequalities $s \not\doteq t$ with $s > t \vee s < t$ and eliminate the resulting disjunction via cvp to obtain a transition without disequalities.

[7] In the case of τ_{dec}, we obtain the same formula τ_δ for *every* model $\sigma \models \tau_{\mathsf{dec}}$, as variables can simply be eliminated by propagating equalities.

- $\mathsf{tp}(\tau, \sigma)$ contains the literal $n > 0$, where $n \in \mathcal{V}$ is a fresh extra variable
- for each literal $\sum_{x \in \mathbf{x}} c_x \cdot (x' - x) + c \bowtie 0$ of τ_{rec}, $\mathsf{tp}(\tau, \sigma)$ contains the literal $\sum_{x \in \mathbf{x}} c_x \cdot (x' - x) + n \cdot c \bowtie 0$
- for each literal $e | \sum_{x \in \mathbf{x}} c_x \cdot (x' - x) + c$ of τ_{rec}, $\mathsf{tp}(\tau, \sigma)$ contains the literal $e | \sum_{x \in \mathbf{x}} c_x \cdot (x' - x) + n \cdot c$

Intuitively, the extra variable n can be thought of as a "loop counter", i.e., when n is instantiated with some constant k, then the literals above approximate the change of variables when $\to_\tau$ is applied k times.

Example 25 (Computing tp (1))**.** Continuing Ex. 24, $\mathsf{tp}(\tau_{\mathsf{dec}}, \sigma)$ contains the literals

$$\begin{aligned} & n > 0 \land w' - w + n \cdot 0 \doteq 0 \land x' - x + n \cdot 1 \doteq 0 \land y' - y + n \cdot 1 \doteq 0 \\ \equiv\ & n > 0 \land w' \doteq w \land x' \doteq x - n \land y' \doteq y - n \end{aligned}$$

for any model $\sigma \models \tau_{\mathsf{dec}}$. Note that in our example, this formula precisely characterizes the change of the variables after n iterations of $\to_{\tau_{\mathsf{dec}}}$. To simplify the formula above, we can propagate the equality $n = x - x'$, resulting in:

$$\begin{aligned} & x - x' > 0 \land w' \doteq w \land y' \doteq y - x + x' \\ \equiv\ & w' \doteq w \land x' < x \land x' - x \doteq y' - y \end{aligned} \tag{7}$$

Compared to τ^+_{dec}, (7) lacks the literal $w \doteq 1$. To incorporate information about the pre- and post-variables (but not about their relation) we conjoin $\mathsf{cvp}(\tau, \sigma, \mathbf{x})$ and $\mathsf{cvp}(\tau, \sigma, \mathbf{x}')$ to $\mathsf{tp}(\tau, \sigma)$.

Example 26 (Computing tp (2))**.** We finish Ex. 25 by conjoining

$$\mathsf{cvp}(\tau_{\mathsf{dec}}, \sigma, \{w, x, y\}) = w \doteq 1 \qquad \text{and} \qquad \mathsf{cvp}(\tau_{\mathsf{dec}}, \sigma, \{w', x', y'\}) = w' \doteq 1$$

to (7), resulting in:

$$w' \doteq w \land x' < x \land x' - x \doteq y' - y \land w \doteq 1 \land w' \doteq 1 \quad \equiv \quad \tau^+_{\mathsf{dec}}$$

Example 27 (Divisibility)**.** To see how tp can handle divisibility predicates, consider the transition

$$\tau := 2|x \land 3|x' - x + 1.$$

Then our approach identifies the recurrent literal $\tau_{\mathsf{rec}} = 3|x' - x + 1$, so that $\mathsf{tp}(\tau, \sigma)$ contains the literal $3|x' - x + n$. To see why we conjoin this literal to $\mathsf{tp}(\tau, \sigma)$, note that $3|x' - x + 1$ and $3|x' - x + n$ are equivalent to $x' - x + 1 \equiv_3 0$ and $x' - x + n \equiv_3 0$, respectively, where "$\equiv_3$" denotes congruence modulo 3. So $e \to^n_\tau e'$ implies $e' - e + n \equiv_3 0$, just like $e \to^n_\pi e'$ implies $e' - e + n = 0$ for $\pi := x' - x + 1 \doteq 0$. Moreover, we have $\mathsf{cvp}(\tau, \sigma, \mathbf{x}) = 2|x$, and thus

$$\mathsf{tp}(\tau, \sigma) = n > 0 \land 3|x' - x + n \land 2|x.$$

We refer to App. A for the straightforward proof of the following theorem:

Theorem 28 (Correctness of tp)**.** *The function* tp *as defined above is a transitive projection.*

5 Proving Unsafety

We now explain how to adapt Alg. 2 for also proving unsafety. Assume that the satisfiability check in Line 5 is successful. Then an error state is reachable from an initial state via the current trace. However, the trace may contain learned relations, so this does not imply unsafety of $\mathcal{T}$. The idea for proving unsafety is to replace learned relations with *accelerated relations* that result from applying *acceleration techniques*.

Definition 29 (Acceleration)**.** A function $\mathsf{accel} : \mathsf{QF}(\Sigma) \to \mathsf{QF}(\Sigma)$ is called an *acceleration technique* if $\to_{\mathsf{accel}(\pi)} \subseteq \to_\pi^+$ for all relational formulas π.

So in contrast to TRL's learned relations, accelerated relations under-approximate transitive closures, and hence they are suitable for proving unsafety. For arithmetical theories, acceleration techniques are well studied [7, 21, 45] (our implementation uses the technique from [21]), and also for the theory of arrays, acceleration is an active field of research [1, 28, 36, 47].

Example 30 (Acceleration)**.** Consider the relation

$$\underbrace{y > 0 \wedge x' \doteq x + 1 \wedge y' \doteq y - 1 \wedge z' \doteq z}_{\tau_>} \vee \underbrace{y \doteq 0 \wedge x' \doteq x \wedge y' \doteq z \wedge z' \doteq z}_{\tau_=} \quad (\tau)$$

with the initial states given by $\psi_{\mathsf{init}} := x \leq 0$ and the error states given by $\psi_{\mathsf{err}} := x \geq 1000$. Assume that TRL obtains the trace $[\tau_>]$ and learns the relation

$$n > 0 \wedge y > 0 \wedge x' \doteq x + n \wedge y' \doteq y - n \wedge z' \doteq z \wedge y' \geq 0. \quad (\tau_>^+)$$

As $\tau_>^+$ characterizes the transitive closure of $\tau_>$ precisely, a suitable acceleration technique also yields $\mathsf{accel}(\tau_>) = \tau_>^+$.

For any sequence of relational formulas $\boldsymbol{\rho} = [\rho_1, \ldots, \rho_k]$, let $\mathsf{compose}(\boldsymbol{\rho})$ be a relational formula such that $\to_{\mathsf{compose}(\boldsymbol{\rho})}$ is the composition of the relations $\to_{\rho_1}, \ldots, \to_{\rho_k}$. So we have $\mathbf{v}_0 \to_{\mathsf{compose}(\boldsymbol{\rho})} \mathbf{v}_k$ if there are $\mathbf{v}_1, \ldots, \mathbf{v}_{k-1} \in \mathcal{C}^d$ such that $\mathbf{v}_0 \to_{\rho_1} \mathbf{v}_1 \to_{\rho_2} \ldots \to_{\rho_k} \mathbf{v}_k$.

Example 31 ($\mathsf{compose}$)**.** We continue Ex. 30. After learning $\tau_>^+$, assume that TRL obtains the trace $[\tau_=, \tau_>^+]$ and learns the relation

$$y \doteq 0 \wedge z > 0 \wedge x' > x \wedge z > y' \wedge y' \geq 0 \wedge z' \doteq z. \quad (\hat{\tau}^+)$$

Note that the transitive closure of the loop $[\tau_=, \tau_>^+]$ cannot be expressed precisely with linear arithmetic, and hence we only obtain the inequations above for x' and y'. Thus, $\hat{\tau}^+$ is an over-approximation of the transitive closure of the loop $[\tau_=, \tau_>^+]$. To compute an under-approximation instead, we first compose $\tau_=$ and $\tau_>^+$, resulting in:

$$y \doteq 0 \wedge n > 0 \wedge z > 0 \wedge x' \doteq x + n \wedge y' \doteq z - n \wedge z' \doteq z \wedge y' \geq 0 \qquad (\check{\tau})$$

Then applying a suitable acceleration technique to $\check{\tau}$ may yield

$$y \doteq 0 \wedge m > 0 \wedge z > 0 \wedge x' \doteq x + m \cdot z \wedge y' \doteq 0 \wedge z' \doteq z. \qquad (\check{\tau}^+)$$

Note that $\check{\tau}^+$ precisely characterizes the transitive closure of $\check{\tau}$ for the case $n \doteq z$, and hence it is a valid under-approximation.

For a learned relation π, we say that $\boldsymbol{\tau}_{\mathsf{loop}}$ *induced* π if we had $\boldsymbol{\tau}_{\mathsf{loop}} = [\tau_s, \ldots, \tau_{s+\ell-1}]$ in Line 12 when π was learned. In other words, $\boldsymbol{\tau}_{\mathsf{loop}}$ is a loop that was discovered on the trace, and when this loop was discovered, the new relation π has been learned. Note that one loop may induce several learned relations, as a learned relation not only depends on the loop that is discovered in Line 12, but also on the model from Line 11.

Example 32 (Induced Relations)**.** In Ex. 30, the learned relation $\tau_>^+$ is induced by $\tau_>$, and in Ex. 31, the learned relation $\check{\tau}^+$ is induced by $[\tau_=, \tau_>^+]$.

Let $\succ$ be the total ordering on the elements of the sequence $\boldsymbol{\pi}$ from Alg. 2 with $\pi_i \succ \pi_j$ iff $i > j$, i.e., the input formula τ is minimal w.r.t. $\succ$, and a learned relation is smaller than all relations that were learned later. Then for sequences of transitions from $\{\mathsf{cvp}(\pi, \sigma) \mid \pi \in \boldsymbol{\pi}, \sigma \models \pi\}$, we define the function ua (which yields an under-approximation) as follows:

$$\mathsf{ua}([\eta_1, \ldots, \eta_k]) := \mathsf{compose}([\mathsf{ua}(\eta_1), \ldots, \mathsf{ua}(\eta_k)])$$

$$\mathsf{ua}(\eta) := \begin{cases} \eta & \text{if } \eta \models \tau \\ \mathsf{accel}(\mathsf{ua}(\boldsymbol{\tau}_{\mathsf{loop}})) & \text{if } \eta \not\models \tau \text{ and} \\ & \quad \text{the loop } \boldsymbol{\tau}_{\mathsf{loop}} \text{ induced } \min_{\succ}\{\pi \in \boldsymbol{\pi} \mid \eta \models \pi\} \end{cases}$$

So ua leaves original transitions unchanged (due to the first case of the definition of $\mathsf{ua}(\eta)$ – recall that τ is the input formula), whereas learned transitions are accelerated (due to the second case of the definition of $\mathsf{ua}(\eta)$). More precisely, for learned transitions η, ua first computes an under-approximation of the loop $\boldsymbol{\tau}_{\mathsf{loop}}$ that induced η, and then it applies acceleration, resulting in an under-approximation of the transitive closure. The reason for applying ua before acceleration is that $\boldsymbol{\tau}_{\mathsf{loop}}$ may again contain learned transitions, which have to be under-approximated first.

Example 33 (Computing Under-Approximations with ua)**.** Continuing Ex. 32, we get:

$\mathsf{ua}([\hat{\tau}^+])$

$= \mathsf{accel}(\mathsf{ua}([\tau_=, \tau_>^+]))$ (as $[\tau_=, \tau_>^+]$ induced $\hat{\tau}^+$, see Ex. 32)

$= \mathsf{accel}(\mathsf{compose}(\mathsf{ua}(\tau_=), \mathsf{ua}(\tau_>^+)))$

$= \mathsf{accel}(\mathsf{compose}(\tau_=, \mathsf{accel}(\mathsf{ua}([\tau_>]))))$ (as $\tau_= \models \tau$ and $\tau_>$ induced $\tau_>^+$, see Ex. 32)

$= \mathsf{accel}(\mathsf{compose}(\tau_=, \mathsf{accel}(\tau_>)))$ (as $\tau_> \models \tau$)

$= \mathsf{accel}(\mathsf{compose}(\tau_=, \tau_>^+))$ (as $\rightarrow_{\tau_>}^+ = \rightarrow_{\tau_>^+}$, see Ex. 30)

$= \mathsf{accel}(\check{\tau})$ (Ex. 31)

$= \check{\tau}^+$ (Ex. 31)

Note that ua is well defined, as the following holds for all $\eta' \in \boldsymbol{\tau}_{\mathsf{loop}}$ in the case $\eta \not\models \tau$:

$$\min_{\succ}\{\pi \in \boldsymbol{\pi} \mid \eta \models \pi\} \succ \min_{\succ}\{\pi \in \boldsymbol{\pi} \mid \eta' \models \pi\}$$

The reason is that $\min_{\succ}\{\pi \in \boldsymbol{\pi} \mid \eta \models \pi\}$ is induced by $\boldsymbol{\tau}_{\mathsf{loop}}$, and thus the elements of $\boldsymbol{\tau}_{\mathsf{loop}}$ are conjunctive variable projections of τ, or of relations that were learned before $\min_{\succ}\{\pi \in \boldsymbol{\pi} \mid \eta \models \pi\}$. Hence, when defining $\mathsf{ua}(\eta)$, the "recursive call" $\mathsf{ua}(\boldsymbol{\tau}_{\mathsf{loop}})$ only refers to formulas with smaller index in $\boldsymbol{\pi}$, i.e., the recursion "terminates".

Our improved version of Alg. 2 can be seen in Alg. 3. Instead of returning $\mathsf{unknown}$, now we first obtain a model σ from the SMT solver, and then we compute the current trace $\boldsymbol{\tau} = \mathsf{trace}_{b-1}(\sigma, \boldsymbol{\pi}) = [\tau_1, \ldots, \tau_{b-1}]$ in Line 6 (note that the trace has length $b-1$, as b was incremented in Line 4). Then we reset the state of the underlying SMT solver in Line 7. Next, we encode the initial states (Line 8), and compute an under-approximation $\mathsf{ua}(\boldsymbol{\tau})$ of the current trace. As $\mathsf{ua}(\boldsymbol{\tau})$ is constructed from original transitions by applying $\mathsf{compose}$ (which is exact) and accel (which yields under-approximations), it also under-approximates $\rightarrow_{\tau}^+$. Finally, the error states are encoded using a suitable variable renaming (Line 10). So if the resulting SMT problem is satisfiable, then an error state is reachable from an initial state using the constructed under-approximation, and hence also via τ. Consequently, unsafe is returned in Line 11. Otherwise, TRL fails by returning $\mathsf{unknown}$ in Line 3.

Example 34 (Proving Unsafety)**.** We finish Ex. 33. A model like

$$[x/y/0, z/1, m/1000,\ x'/1000, y'/0, z'/1]$$

satisfies $\psi_{\mathsf{init}} \wedge \check{\tau}^+ \wedge \psi_{\mathsf{err}}[\mathbf{x}/\mathbf{x}'] = x \leq 0 \wedge \check{\tau}^+ \wedge x' \geq 1000$. Thus, the safety problem from Ex. 30 is unsafe.

We conclude this section with the soundness theorem for Alg. 3.

Theorem 35. *If Alg. 3 returns* safe*, then* $\mathcal{T}$ *is safe. If Alg. 3 returns* unsafe*, then* $\mathcal{T}$ *is unsafe.*

Proof For safety, the claim follows from Thm. 22. For unsafety, note that the result of ua is constructed from original transitions using the operations $\mathsf{compose}$ (which is exact) and accel

Algorithm 3: TRL with support for unsafety

Input: a safety problem $\mathcal{T} = (\psi_{\mathsf{init}}, \tau, \psi_{\mathsf{err}})$
Output: safe, unsafe, or unknown

1 $b \leftarrow 0$; $\boldsymbol{\pi} \leftarrow [\tau]$; BLOCKED $\leftarrow \varnothing$
2 $\mathsf{add}(\mu_1(\psi_{\mathsf{init}}))$ // encode the initial states
3 **while** $\top$ // main loop
4 $\quad b$++; $\mathsf{push}()$; $\mathsf{add}(\mu_b(\psi_{\mathsf{err}}))$ // encode the error states
5 $\quad$ **if** check_sat() **do** // check their reachability with τ and learned relations
6 $\quad\quad \sigma \leftarrow \mathsf{model}()$; $\boldsymbol{\tau} \leftarrow \mathsf{trace}_{b-1}(\sigma, \boldsymbol{\pi})$ // build trace from current model
7 $\quad\quad$ reset_solver() // remove all assertions from the SMT solver
8 $\quad\quad \mathsf{add}(\psi_{\mathsf{init}})$ // encode the initial states
9 $\quad\quad \mathsf{add}(\mathsf{ua}(\boldsymbol{\tau}))$ // encode an under-approximation of the trace
10 $\quad\quad \mathsf{add}(\psi_{\mathsf{err}}[\mathbf{x}/\mathbf{x}'])$ // encode the error states
11 $\quad\quad$ **if** check_sat() **do return** unsafe // check their reachability with $\mathsf{ua}(\boldsymbol{\tau})$
12 $\quad\quad$ **else return** unknown
13 $\quad$ **else** pop()
14 $\quad \mathsf{push}()$ // add backtracking point
15 $\quad$ **if** $b > 1$ **do** $\mathsf{add}(\overset{(b)}{x}_{\mathsf{id}} \doteq 1 \vee \overset{(b)}{x}_{\mathsf{id}} \neq \overset{(b-1)}{x}_{\mathsf{id}})$ // encode transitivity
16 $\quad \mathsf{add}(\mu_b(\bigvee_{n=1}^{|\boldsymbol{\pi}|}(\pi_n \wedge x_{\mathsf{id}} \doteq n)))$ // encode $\rightarrow_\tau$ and learned relations
17 $\quad \mathsf{add}(\bigwedge_{(b,\pi)\in \text{BLOCKED}} \pi)$ // add blocking clauses for this b
18 $\quad$ **if** $\neg$check_sat() **do return** safe // check if the search space is exhausted
19 $\quad \sigma \leftarrow \mathsf{model}()$; $\boldsymbol{\tau} \leftarrow \mathsf{trace}_b(\sigma, \boldsymbol{\pi})$ // build trace from current model
20 $\quad$ **if** $[\tau_s, \ldots, \tau_{s+\ell-1}]$ *is a loop* **do** // search loop
21 $\quad\quad \sigma_{\mathsf{loop}} \leftarrow [x/\sigma(\mu_{s,\ell}(x)) \mid x \in \mathbf{x} \cup \mathbf{x}']$ // build the valuation for the loop
22 $\quad\quad$ **if** *no* $\pi \in \mathsf{tail}(\boldsymbol{\pi})$ *is consistent with* σ_{loop} **do** // redundancy check
23 $\quad\quad\quad \tau_{\mathsf{loop}} \leftarrow \mu_{s,\ell}^{-1}(\bigwedge_{i=s}^{s+\ell-1} \mu_i(\tau_i))$ // build the loop
24 $\quad\quad\quad \boldsymbol{\pi} \leftarrow \boldsymbol{\pi} :: \mathsf{tp}(\tau_{\mathsf{loop}}, \sigma \circ \mu_{s,\ell})$ // learn relation
25 $\quad\quad$ let $\pi \in \mathsf{tail}(\boldsymbol{\pi})$ and $\overline{\sigma} \supseteq \sigma_{\mathsf{loop}}$ s.t. $\overline{\sigma} \models \pi$ // pick suitable learned relation
26 $\quad\quad$ BLOCKED.$\mathsf{add}(s + \ell - 1, \mathsf{blocking_clause}(s, \ell, \pi, \overline{\sigma}))$ // block the loop
27 $\quad\quad$ **while** $b \geq s$ { pop(); b-- } // backtrack to the start of the loop

(which yields an under-approximation of $\rightarrow_\tau^+$). Thus, we have $\rightarrow_\tau^+ \subseteq \rightarrow_{\mathsf{ua}(\boldsymbol{\tau})}$. Hence, unsafety of $(\psi_{\mathsf{init}}, \mathsf{ua}(\boldsymbol{\tau}), \psi_{\mathsf{err}})$ implies unsafety of $(\psi_{\mathsf{init}}, \tau, \psi_{\mathsf{err}})$. If satisfiability of the underlying SMT problem is proven in Line 11, then $\psi_{\mathsf{init}} \wedge \mathsf{ua}(\boldsymbol{\tau}) \wedge \psi_{\mathsf{err}}[\mathbf{x}/\mathbf{x}']$ is satisfiable, which implies unsafety of $(\psi_{\mathsf{init}}, \mathsf{ua}(\boldsymbol{\tau}), \psi_{\mathsf{err}})$. Thus, the claim follows. □

6 Interleaving Forward and Backward Analysis

One crucial advantage of interpolation-based model checking algorithms over TRL is that they are driven by the given safety property. All of them make use of potentially spurious counterexamples. If such an example is indeed a counterexample, then they return unsafe. Otherwise, they extract an interpolant from the proof that the counterexample is spurious, and then they use this interpolant to refine the analysis.

In contrast, TRL's core ingredient, transitive projection, does not take the safety property into account. Instead, it only considers the current model to compute an

approximation of the transitive closure of a loop. However, there are cases where considering the safety property simplifies the safety proof dramatically.

Example 36 (Property-Directed Reasoning)**.** Consider the safety problem consisting of the initial states $x > 0$, the relational formula

$$\tau := x' \doteq x + y \wedge y' \doteq y \wedge \bigvee_{i=1}^{42} y = i,$$

and the error states $x \leq 0 \wedge y = 1$. Without taking the error states into account, TRL may learn one relation

$$x' = x + i \cdot n \wedge y' = y = i \wedge n > 0$$

for each possible value of $i \in \{1, \ldots, 42\}$, i.e., it may learn up to 42 relations. However, by considering the error states, it is obvious that $i = 1$ is the only relevant case.

A simple way to handle cases like Ex. 36 is to *invert* the given safety problem.

Definition 37 (Inverted Safety Problem)**.** *Inverting* the safety problem $(\psi_{\mathsf{init}}, \tau, \psi_{\mathsf{err}})$ yields $(\psi_{\mathsf{err}}, \tau[\mathbf{x}/\mathbf{x}', \mathbf{x}'/\mathbf{x}], \psi_{\mathsf{init}})$.

So by analyzing the inverted safety problem, one checks whether an initial state is *backward-reachable* from an error state, which is clearly equivalent to unsafety.

Lemma 38 (Inversion Preserves Safety)**.** *A safety problem is safe if and only if its inverted safety problem is safe.*

Proof Let $(\psi_{\mathsf{init}}, \tau, \psi_{\mathsf{err}})$ be an arbitrary safety problem and let $\overline{\tau} = \tau[\mathbf{x}/\mathbf{x}', \mathbf{x}'/\mathbf{x}]$. As inversion is self-inverse (i.e., the inverse of $(\psi_{\mathsf{err}}, \overline{\tau}, \psi_{\mathsf{init}})$ is again $(\psi_{\mathsf{init}}, \tau, \psi_{\mathsf{err}})$), it suffices to show that unsafety of $(\psi_{\mathsf{init}}, \tau, \psi_{\mathsf{err}})$ implies unsafety of $(\psi_{\mathsf{err}}, \overline{\tau}, \psi_{\mathsf{init}})$. Assume that $(\psi_{\mathsf{init}}, \tau, \psi_{\mathsf{err}})$ is unsafe. Then there is a $k \in \mathbb{N}$ and $\mathbf{v}_0, \ldots, \mathbf{v}_k \in \mathcal{C}^d$ such that $[\mathbf{x}/\mathbf{v}_0] \models \psi_{\mathsf{init}}$, $\mathbf{v}_0 \to_\tau \mathbf{v}_1 \to_\tau \ldots \to_\tau \mathbf{v}_k$, and $[\mathbf{x}/\mathbf{v}_k] \models \psi_{\mathsf{err}}$. As we clearly have $\mathbf{v} \to_\tau \mathbf{v}'$ iff $\mathbf{v}' \to_{\overline{\tau}} \mathbf{v}$, this implies $\mathbf{v}_k \to_{\overline{\tau}} \mathbf{v}_{k-1} \to_{\overline{\tau}} \ldots \to_{\overline{\tau}} \mathbf{v}_0$. Hence, $(\psi_{\mathsf{err}}, \overline{\tau}, \psi_{\mathsf{init}})$ is unsafe, so the claim follows. □

Example 39 (Running TRL Backward)**.** Reconsider the safety problem from Ex. 36, and let $(\psi_{\mathsf{err}}, \overline{\tau}, \psi_{\mathsf{init}})$ be its inverted safety problem. By running TRL backward instead of forward, we obtain a trace of length one with the only element

$$\overline{\tau}_1 := x' \doteq x - y \wedge y' \doteq y \wedge y = 1,$$

which is a loop. Transitive projection yields

$$x' \doteq x - n \wedge y' = y = 1 \wedge n > 0,$$

which precisely characterizes the transitive closure of $\overline{\tau}$ for the case $y = 1$. After backtracking, adding a suitable blocking clause for the first step, and encoding that learned relations must not be used twice in a row, the underlying SMT problem becomes unsatisfiable, i.e., TRL returns safe. So by inverting the safety problem, it suffices to learn 1 instead of up to 42 relations.

To combine forward and backward analysis, there are several options. The most obvious one is to run both analyses in parallel. Alternatively, one can integrate both analyses more tightly, such that they can share learned relations. So if, e.g., the backward analysis learns the relation π, then $\pi[\mathbf{x}/\mathbf{x}', \mathbf{x}'/\mathbf{x}]$ is added to the learned relations of the forward analysis. However, so far our tool LoAT does not use parallelization, and sharing learned relations turned out to be disadvantageous. The reason is that if, e.g., the forward analysis learns a relation that is too imprecise and prevents it from proving safety, then this relation may also prevent the backward analysis from proving safety, and thus both analyses fail.

Thus, our implementation interleaves both analyses in a single process, but they do not have any shared state. So first, the forward analysis executes one iteration of the main loop of Alg. 3, then the backward analysis executes one iteration, then the forward analysis executes another iteration, etc. As soon as one analysis returns safe or unsafe, or both analyses returned unknown, our implementation terminates. Alternatively, the user can choose to just run the forward or backward analysis.

7 Related Work

Most state-of-the-art infinite state model checking algorithms rely on *interpolation* [10, 37, 38, 41, 43, 49, 50, 58]. As discussed in Sect. 1, these techniques are very powerful, but also fragile, as interpolation depends on the implementation details of the underlying SMT solver. In contrast, robustness of TRL depends on the underlying transitive projection. Our implementation for linear integer arithmetic is a variation of the recurrence analysis from [20], which is very robust in our experience. In particular, our recurrence analysis does not require an SMT solver.

Some state-of-the-art solvers like Golem [6] and our own tool LoAT also implement *k-induction* [56], which is more robust than interpolation-based techniques. However, k-induction can only prove k-inductive properties, and our experiments (see Sect. 8) show that it is not competitive with other state-of-the-art approaches.

Currently, the most powerful model checking algorithm is *Spacer with global guidance* (GSpacer). In GSpacer, interpolation is optional, and thus it is more robust than Spacer (without global guidance) and other interpolation-based techniques. GSpacer is part of the IC3/PDR family of model checking algorithms [8], which differ fundamentally from BMC-based approaches like TRL: While the latter unroll the transition relation step by step, IC3/PDR proves global properties by combining local reasoning about a single step with induction, interpolation, or other mechanisms like global guidance. This fundamental difference can also be seen in our evaluation (Sect. 8), where both GSpacer and TRL find many unique safety proofs. A main advantage of IC3/PDR over BMC-based techniques is scalability, as unrolling the transition relation can quickly

become a bottleneck for large systems. However, TRL mitigates this drawback, as it always backtracks when the trace contains a loop. Hence, TRL rarely needs to consider long unrollings.

For program verification, there are several techniques that use *transition invariants* or closely related concepts like *loop summarization* or *acceleration* [3, 7, 20, 23, 26, 45, 46, 52, 57].

Acceleration-based techniques [1, 3, 7, 23, 26, 28, 36, 46, 47] try to compute transitive closures of loops exactly, or to under-approximate them. However, even for simple loops like `while(...) x = x + y`, meaningful under-approximations cannot be expressed in a decidable logic. For example, for the loop above, the non-linear formula $\exists n.\ n > 0 \wedge x' \doteq x + n \cdot y$ would be a precise characterization of the transitive closure. Thus, these approaches either only use acceleration for loops where decidable logics are sufficient, or they resort to undecidable logics, which is challenging for the underlying SMT solver. TRL is not restricted to under-approximations, and hence it can approximate arbitrary loops without resorting to undecidable logics.

As mentioned in Sect. 1, TRL has been inspired by our work on accelerated bounded model checking (ABMC) [26]. For the reasons explained above, TRL is more powerful than ABMC for proving safety (see also Sect. 8), but it does not subsume ABMC for that purpose. The reason is that ABMC's acceleration sometimes yields non-linear relations, whereas TRL only learns linear relations. On the other hand, ABMC is more powerful than TRL for proving unsafety, as TRL only tries to construct a counterexample from a single trace, whereas ABMC explores many different traces. Hence, both approaches are orthogonal in practice. Apart from that, a major difference is that ABMC uses blocking clauses only sparingly, which is disadvantageous for proving safety. The reason is that blocking clauses reduce the search space for counterexamples, and to prove safety, this search space has to be exhausted, eventually. However, using them whenever possible is disadvantageous as well, as it blows up the underlying SMT problem unnecessarily. Hence, TRL uses them on demand instead. Other important conceptual differences include TRL's use of model based projection, which cannot be used by ABMC, as acceleration yields formulas in theories without effective quantifier elimination. Moreover, TRL uses backtracking, which is useful for proving safety as it keeps the underlying SMT problem small, but it would slow down the discovery of counterexamples in ABMC.

Our tool LoAT also implements ADCL [23], which embeds acceleration into a depth-first search for counterexamples (whereas ABMC and TRL perform breadth-first search). Thus, TRL is conceptually closer to ABMC, and for proving unsafety, ABMC is superior to ADCL (see [26]). However, as witnessed by the results of the annual *Termination Competition* [32], ADCL is currently the most powerful technique for proving non-termination of transition systems (see [24]).

In contrast to acceleration-based techniques, recurrence analysis and loop summarization [20, 45, 52, 57] over-approximate transitive closures of loops. TRL is not restricted to over-approximations, which allows for increased precision, as our transitive projections can perform a case analysis based on the provided model. Note that this cannot be done in a framework that requires over-approximations, as `tp` cannot be used to compute over-approximations, in general (see Remark 16).

The techniques from [52,57] express over-approximations using (variations of) *vector addition systems with resets (VASRs)*, which are less expressive than our transitive projections, as the latter may yield arbitrary relational formulas. However, [52, 57] ensure that the computed approximations are the best VASR-approximations. Our approach does not provide any such guarantees, so both approaches are orthogonal.

Apart from these techniques, transition invariants have mainly been used in the context of termination analysis and temporal verification [35, 44, 54, 59, 61]. We expect that our approach can be adapted to these problems, but it is unclear whether it would be competitive.

8 Experiments and Conclusion

We implemented TRL in our tool LoAT [22]. Our implementation uses the SMT solvers Yices [16] (for linear arithmetic), Z3 [51] (for non-linear, polynomial arithmetic), and SwInE [25] (for non-linear arithmetic also involving exponentiation). Currently, our implementation is restricted to input problems with linear integer arithmetic only, but despite that, LoAT's acceleration techniques (see Sect. 5) may yield non-linear formulas. We evaluated our approach on all examples from the category LIA-Lin (linear CHCs with linear integer arithmetic) from the CHC competitions 2023 and 2024 [12] (excluding duplicates),[8] resulting in a collection of 626 problems from numerous applications like verification of C, Rust, Java, and higher-order programs, and regression verification of LLVM programs, see [15, 19] for details. By using CHCs as input format, our approach can be used by any CHC-based tool like Korn [18] and SeaHorn [34] for C and C++ programs, JayHorn for Java programs [39], HornDroid for Android [11], RustHorn for Rust programs [48], and SmartACE [60] and SolCMC [2] for Solidity.

We compared TRL with the techniques of leading CHC solvers. More precisely, we evaluated the following configurations:

LoAT We used LoAT's implementations of TRL, ABMC [26] (LoAT ABMC), and *k-induction* [56] (LoAT KIND). For TRL, we used three variants: LoAT TRL Forward performs a forward analysis, LoAT TRL Backward performs a backward analysis, and LoAT TRL Interleaved performs an interleaved analysis, see Sect. 6 for details.

Z3 [51] We used Z3 4.13.3, where we evaluated its implementations of the Spacer [41] (Z3 Spacer) and GSpacer [43] (Z3 GSpacer) algorithms, and of BMC (Z3 BMC).

Golem [6] We used Golem 0.6.2, where we evaluated its implementations of *transition power abstraction* [10] (Golem TPA), *interpolation based model checking* [49] (Golem IMC), *lazy abstraction with interpolants* [50] (Golem LAWI), *predicate abstraction* [33] with *CEGAR* [13] (Golem PA), *property directed k-induction* [38] (Golem PDKIND), Spacer (Golem Spacer), and BMC (Golem BMC). Note that Golem's implementation of k-induction only applies to linear CHC problems with a specific structure (just one fact, one recursive rule, and one query), so we excluded it from our evaluation.

[8] While these competitions ran on subsets of the collection of examples for the CHC competition, the CHC competition 2025 ran on the entire collection. However, using the entire collection exceeds the computational resources that we have access to for our evaluation.

Eldarica [37] We used the default configuration of Eldarica 2.1, which is based on predicate abstraction and CEGAR.

We ran our experiments on CLAIX-2023-HPC nodes of the RWTH University High Performance Computing Cluster[9] with a memory limit of 10560 MiB (≈ 11GB) and a timeout of 300 s per example. Note that the techniques from Sect. 5 and 6 were already enabled in the evaluation from [27], i.e., the configuration LoAT TRL Interleaved corresponds to the configuration LoAT TRL from [27]. So compared to the evaluation from [27], we consider the following additional configurations:

LoAT: TRL Backward, TRL Forward, ABMC, KIND
Z3: BMC
Golem: LAWI, TPA, PA, BMC

	✓	safe		unsafe	
		✓	!	✓	!
LoAT TRL Interleaved	386	267	16(19)	119	1(9)
Z3 GSpacer	373	287	16(16)	86	3(3)
LoAT TRL Backward	349	241	–	108	–
Golem Spacer	340	231	–	109	–
LoAT TRL Forward	340	224	–	116	–
Z3 Spacer	339	236	–	103	–
Golem IMC	318	215	2(2)	103	0(0)
Golem PDKIND	315	224	2(2)	91	0(0)
Eldarica	298	219	7(7)	79	0(0)
Golem LAWI	290	182	0(0)	108	1(1)
Golem TPA	289	170	1(2)	119	2(4)
LoAT ABMC	272	142	1(–)	130	2(–)
Golem PA	253	160	–	93	–
LoAT KIND	245	133	0(0)	112	0(0)
Golem BMC	145	34	0(0)	111	0(0)
Z3 BMC	145	32	–	113	–

Fig. 1: Results of our Evaluation

The results can be seen in the table above. The first column marked with ✓ contains the number of solved instances (i.e., all remaining instances could not be solved by the respective configuration). The columns with **!** show the number of unique proofs, i.e., the number of examples that could only be solved by the corresponding configuration. Such a comparison only makes sense if just one implementation of each algorithm is considered. For instance, GSpacer is an improved version of Spacer, so Z3 GSpacer,

[9] https://help.itc.rwth-aachen.de/service/rhr4fjjutttf

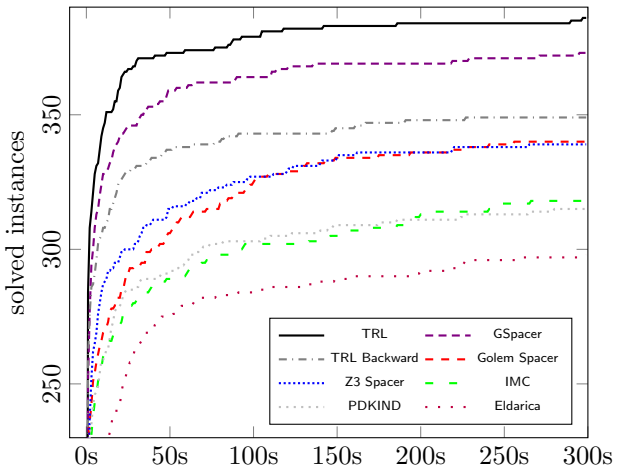


Fig. 2: Comparison of Runtimes

Z3 Spacer, and Golem Spacer work well on the same class of examples. Hence, if all of them were considered, they would find very few unique proofs. The same is true for the three variants of TRL, Eldarica and Golem PA, as well as Golem BMC and Z3 BMC. Thus, for **!** we disregarded TRL Forward, TRL Backward, Z3 Spacer, Golem Spacer, Golem PA, and Z3 BMC.

The numbers in parentheses in the columns marked with **!** show the number of unique proofs when also disregarding LoAT ABMC. We included this number since TRL was inspired by ABMC. So analogously to GSpacer and Spacer, one could consider TRL to be an improved version of ABMC for proving safety.

The table shows that TRL is highly competitive: Overall, it solves the most instances. Even more importantly, it finds many unique proofs, i.e., it is orthogonal to existing model checking algorithms, and hence it improves the state of the art. The fact that LoAT TRL Interleaved significantly outperforms LoAT TRL Forward and LoAT TRL Backward also shows the importance of Sect. 6.

Regarding safe instances, TRL is only outperformed by GSpacer. Regarding unsafe instances, TRL is only outperformed by ABMC, which has specifically been developed for finding (deep) counterexamples, whereas TRL's main purpose is proving safety. This shows the effectiveness of the approach from Sect. 5.

The plot above shows how many instances were solved within 300 s, where we only include eight out of the nine best configurations for readability (we excluded TRL Forward, which performs very similar to TRL Backward). It shows that TRL is highly competitive, not only in terms of solved examples, but also in terms of runtime.

Our implementation is open-source and available on Github [30]. For the sources, a pre-compiled binary, and more information on our evaluation, we refer to [29].

Conclusion

We presented *Transitive Relation Learning (TRL)*, a powerful model checking algorithm for infinite state systems. Instead of searching for invariants, TRL adds transitive relations to the analyzed system until its *diameter* (the number of steps that is required to cover all reachable states) becomes finite, which facilitates a safety proof. As it does not search for invariants, TRL does not need interpolation, which is in contrast to most other state-of-the-art techniques. Nevertheless, our evaluation shows that TRL is highly competitive with interpolation-based approaches. Moreover, not using interpolation allows us to avoid the well-known fragility of interpolation-based approaches. Integrating *acceleration techniques* into TRL also yields a competitive technique for proving *un*safety. Finally, combining *forward* and *backward* reasoning improves the power of our approach even further.

In future work, we plan to support other theories like reals, bitvectors, and arrays, and we will investigate an extension to temporal verification.

References

[1] Alberti, F., Ghilardi, S., Sharygina, N.: Definability of accelerated relations in a theory of arrays and its applications. In: FroCoS '13. pp. 23–39. LNCS 8152 (2013). https://doi.org/10.1007/978-3-642-40885-4_3

[2] Alt, L., Blicha, M., Hyvärinen, A.E.J., Sharygina, N.: SolCMC: Solidity compiler's model checker. In: CAV '22. pp. 325–338. LNCS 13371 (2022). https://doi.org/10.1007/978-3-031-13185-1_16

[3] Bardin, S., Finkel, A., Leroux, J., Schnoebelen, P.: Flat acceleration in symbolic model checking. In: ATVA '05. pp. 474–488. LNCS 3707 (2005). https://doi.org/10.1007/11562948_35

[4] Barrett, C., Fontaine, P., Tinelli, C.: The Satisfiability Modulo Theories Library (SMT-LIB). `www.SMT-LIB.org` (2016)

[5] Biere, A., Cimatti, A., Clarke, E.M., Strichman, O., Zhu, Y.: Bounded model checking. Advances in Computers **58**, 117–148 (2003). https://doi.org/10.1016/S0065-2458(03)58003-2

[6] Blicha, M., Britikov, K., Sharygina, N.: The Golem Horn solver. In: CAV '23. pp. 209–223. LNCS 13965 (2023). https://doi.org/10.1007/978-3-031-37703-7_10

[7] Bozga, M., Iosif, R., Konečný, F.: Fast acceleration of ultimately periodic relations. In: CAV '10. pp. 227–242. LNCS 6174 (2010). https://doi.org/10.1007/978-3-642-14295-6_23

[8] Bradley, A.R.: SAT-based model checking without unrolling. In: VMCAI '11. pp. 70–87. LNCS 6538 (2011). https://doi.org/10.1007/978-3-642-18275-4_7

[9] Bremner, D.: Incremental convex hull algorithms are not output sensitive. Discret. Comput. Geom. **21**(1), 57–68 (1999). https://doi.org/10.1007/PL00009410

[10] Britikov, K., Blicha, M., Sharygina, N., Fedyukovich, G.: Reachability analysis for multiloop programs using transition power abstraction. In: FM '24. pp. 558–576. LNCS 14933 (2024). https://doi.org/10.1007/978-3-031-71162-6_29

[11] Calzavara, S., Grishchenko, I., Maffei, M.: HornDroid: Practical and sound static analysis of Android applications by SMT solving. In: EuroS&P '16. pp. 47–62.

IEEE (2016). https://doi.org/10.1109/EuroSP.2016.16
[12] CHC Competition, https://chc-comp.github.io
[13] Clarke, E.M., Grumberg, O., Jha, S., Lu, Y., Veith, H.: Counterexample-guided abstraction refinement. In: CAV '00. pp. 154–169. LNCS 1855 (2000). https://doi.org/10.1007/10722167_15
[14] Cooper, D.C.: Theorem proving in arithmetic without multiplication. Machine Intelligence **7**, 91–99 (1972)
[15] De Angelis, E., Govind V K, H.: CHC-COMP 2023: Competition report (2023), https://chc-comp.github.io/2023/CHC_COMP_2023_Competition_Report.pdf
[16] Dutertre, B.: Yices 2.2. In: CAV '14. pp. 737–744. LNCS 8559 (2014). https://doi.org/10.1007/978-3-319-08867-9_49
[17] Enderton, H.B.: A Mathematical Introduction to Logic. Academic Press (1972)
[18] Ernst, G.: Loop verification with invariants and contracts. In: VMCAI '22. pp. 69–92. LNCS 13182 (2022). https://doi.org/10.1007/978-3-030-94583-1_4
[19] Ernst, G., Morales, J.F.: CHC-COMP 2024: Competition report (2024), https://chc-comp.github.io/2024/CHC-COMP%202024%20Report%20-%20HCSV.pdf
[20] Farzan, A., Kincaid, Z.: Compositional recurrence analysis. In: FMCAD '15. pp. 57–64 (2015). https://doi.org/10.1109/FMCAD.2015.7542253
[21] Frohn, F.: A calculus for modular loop acceleration. In: TACAS '20. pp. 58–76. LNCS 12078 (2020). https://doi.org/10.1007/978-3-030-45190-5_4
[22] Frohn, F., Giesl, J.: Proving non-termination and lower runtime bounds with LoAT (system description). In: IJCAR '22. pp. 712–722. LNCS 13385 (2022). https://doi.org/10.1007/978-3-031-10769-6_41
[23] Frohn, F., Giesl, J.: ADCL: Acceleration Driven Clause Learning for constrained Horn clauses. In: SAS '23. pp. 259–285. LNCS 14284 (2023). https://doi.org/10.1007/978-3-031-44245-2_13
[24] Frohn, F., Giesl, J.: Proving non-termination by Acceleration Driven Clause Learning. In: CADE '23. pp. 220–233. LNCS 14132 (2023). https://doi.org/10.1007/978-3-031-38499-8_13
[25] Frohn, F., Giesl, J.: Satisfiability modulo exponential integer arithmetic. In: IJCAR '24. pp. 344–365. LNCS 14739 (2024). https://doi.org/10.1007/978-3-031-63498-7_21
[26] Frohn, F., Giesl, J.: Integrating loop acceleration into bounded model checking. In: FM '24. pp. 73–91. LNCS 14933 (2024). https://doi.org/10.1007/978-3-031-71162-6_4
[27] Frohn, F., Giesl, J.: Infinite state model checking by learning transitive relations. In: CADE '25. pp. 468–489. LNCS (2025). https://doi.org/10.1007/978-3-031-99984-0_25, also available from https://loat-developers.github.io/trl-eval/paper.pdf
[28] Frohn, F., Giesl, J.: Accelerating loops with arrays. In: IJCAR '26. LNCS (2026), to appear. Also available as arXiv:2605.19499
[29] Frohn, F., Giesl, J.: Evaluation of "Infinite State Model Checking by Learning Transitive Relations" (2026), https://loat-developers.github.io/trl-eval/journal
[30] Frohn, F., Giesl, J.: LoAT website, https://loat-developers.github.io/LoAT
[31] Genov, B.: The Convex Hull Problem in Practice: Improving the Running Time

of the Double Description Method. Ph.D. thesis, Bremen University, Germany (2015), https://media.suub.uni-bremen.de/handle/elib/833
[32] Giesl, J., Rubio, A., Sternagel, C., Waldmann, J., Yamada, A.: The termination and complexity competition. In: TACAS '19. pp. 156–166. LNCS 11429 (2019). https://doi.org/10.1007/978-3-030-17502-3_10
[33] Graf, S., Saïdi, H.: Construction of abstract state graphs with PVS. In: CAV '97. pp. 72–83. LNCS 1254 (1997). https://doi.org/10.1007/3-540-63166-6_10
[34] Gurfinkel, A., Kahsai, T., Komuravelli, A., Navas, J.A.: The SeaHorn verification framework. In: CAV '15. pp. 343–361. LNCS 9206 (2015). https://doi.org/10.1007/978-3-319-21690-4_20
[35] Heizmann, M., Jones, N.D., Podelski, A.: Size-change termination and transition invariants. In: Cousot, R., Martel, M. (eds.) SAS '10. pp. 22–50. LNCS 6337 (2010). https://doi.org/10.1007/978-3-642-15769-1_4
[36] Henzinger, T.A., Hottelier, T., Kovács, L., Rybalchenko, A.: Aligators for arrays (tool paper). In: LPAR '10. pp. 348–356. LNCS 6397 (2010). https://doi.org/10.1007/978-3-642-16242-8_25
[37] Hojjat, H., Rümmer, P.: The Eldarica Horn solver. In: FMCAD '18. pp. 1–7 (2018). https://doi.org/10.23919/FMCAD.2018.8603013
[38] Jovanovic, D., Dutertre, B.: Property-directed k-induction. In: FMCAD '16. pp. 85–92 (2016). https://doi.org/10.1109/FMCAD.2016.7886665
[39] Kahsai, T., Rümmer, P., Sanchez, H., Schäf, M.: JayHorn: A framework for verifying Java programs. In: CAV '16. pp. 352–358. LNCS 9779 (2016). https://doi.org/10.1007/978-3-319-41528-4_19
[40] Kincaid, Z., Breck, J., Boroujeni, A.F., Reps, T.W.: Compositional recurrence analysis revisited. In: PLDI '17. pp. 248–262 (2017). https://doi.org/10.1145/3062341.3062373
[41] Komuravelli, A., Gurfinkel, A., Chaki, S.: SMT-based model checking for recursive programs. Formal Methods Syst. Des. **48**(3), 175–205 (2016). https://doi.org/10.1007/s10703-016-0249-4
[42] Konečný, F.: PTIME computation of transitive closures of octagonal relations. In: TACAS '16. pp. 645–661. LNCS 9636 (2016). https://doi.org/10.1007/978-3-662-49674-9_42
[43] Krishnan, H.G.V., Chen, Y., Shoham, S., Gurfinkel, A.: Global guidance for local generalization in model checking. In: CAV '20. pp. 101–125. LNCS 12225 (2020). https://doi.org/10.1007/978-3-030-53291-8_7
[44] Kroening, D., Sharygina, N., Tsitovich, A., Wintersteiger, C.M.: Termination analysis with compositional transition invariants. In: CAV '10. pp. 89–103. LNCS 6174 (2010). https://doi.org/10.1007/978-3-642-14295-6_9
[45] Kroening, D., Sharygina, N., Tonetta, S., Tsitovich, A., Wintersteiger, C.M.: Loop summarization using state and transition invariants. Formal Methods Syst. Des. **42**(3), 221–261 (2013). https://doi.org/10.1007/S10703-012-0176-Y
[46] Kroening, D., Lewis, M., Weissenbacher, G.: Proving safety with trace automata and bounded model checking. In: FM '15. pp. 325–341. LNCS 9109 (2015). https://doi.org/10.1007/978-3-319-19249-9_21
[47] Kroening, D., Lewis, M., Weissenbacher, G.: Under-approximating loops in C

programs for fast counterexample detection. Formal Methods Syst. Des. **47**(1), 75–92 (2015). https://doi.org/10.1007/s10703-015-0228-1
[48] Matsushita, Y., Tsukada, T., Kobayashi, N.: RustHorn: CHC-based verification for Rust programs. ACM Trans. Program. Lang. Syst. **43**(4), 15:1–15:54 (2021). https://doi.org/10.1145/3462205
[49] McMillan, K.L.: Interpolation and SAT-based model checking. In: CAV '03. pp. 1–13. LNCS 2725 (2003). https://doi.org/10.1007/978-3-540-45069-6_1
[50] McMillan, K.L.: Lazy abstraction with interpolants. In: CAV '06. pp. 123–136. LNCS 4144 (2006). https://doi.org/10.1007/11817963_14
[51] de Moura, L., Bjørner, N.: Z3: An efficient SMT solver. In: TACAS '08. pp. 337–340. LNCS 4963 (2008). https://doi.org/10.1007/978-3-540-78800-3_24
[52] Pimpalkhare, N., Kincaid, Z.: Semi-linear VASR for over-approximate semi-linear transition system reachability. In: RP '24. pp. 154–166. LNCS 15050 (2024). https://doi.org/10.1007/978-3-031-72621-7_11
[53] Podelski, A., Rybalchenko, A.: Transition invariants. In: LICS '04. pp. 32–41 (2004). https://doi.org/10.1109/LICS.2004.1319598
[54] Podelski, A., Rybalchenko, A.: Transition invariants and transition predicate abstraction for program termination. In: TACAS '11. pp. 3–10. LCNS 6605 (2011). https://doi.org/10.1007/978-3-642-19835-9_2
[55] Promies, V., Ábrahám, E.: A divide-and-conquer approach to variable elimination in linear real arithmetic. In: FM '24. pp. 131–148. LNCS 14933 (2024). https://doi.org/10.1007/978-3-031-71162-6_7
[56] Sheeran, M., Singh, S., Stålmarck, G.: Checking safety properties using induction and a SAT-solver. In: FMCAD '00. pp. 108–125. LNCS 1954 (2000). https://doi.org/10.1007/3-540-40922-X_8
[57] Silverman, J., Kincaid, Z.: Loop summarization with rational vector addition systems. In: CAV '19. pp. 97–115. LNCS 11562 (2019). https://doi.org/10.1007/978-3-030-25543-5_7
[58] Solanki, M., Chatterjee, P., Lal, A., Roy, S.: Accelerated bounded model checking using interpolation based summaries. In: TACAS '24. pp. 155–174. LNCS 14571 (2024). https://doi.org/10.1007/978-3-031-57249-4_8
[59] Tsitovich, A., Sharygina, N., Wintersteiger, C.M., Kroening, D.: Loop summarization and termination analysis. In: TACAS '11. pp. 81–95. LNCS 6605 (2011). https://doi.org/10.1007/978-3-642-19835-9_9
[60] Wesley, S., Christakis, M., Navas, J.A., Trefler, R.J., Wüstholz, V., Gurfinkel, A.: Verifying Solidity smart contracts via communication abstraction in SmartACE. In: VMCAI '22. pp. 425–449. LNCS 13182 (2022). https://doi.org/10.1007/978-3-030-94583-1_21
[61] Zuleger, F.: Inductive termination proofs with transition invariants and their relationship to the size-change abstraction. In: SAS '18. pp. 423–444. LNCS 11002 (2018). https://doi.org/10.1007/978-3-319-99725-4_25

A Appendix

A.1 Proof of Thm. 22

Theorem 22. *If Alg. 2 returns* safe*, then $\mathcal{T}$ is safe.*

Proof Consider the SMT problem that is checked in Line 10. In the mth iteration (starting with $m = 1$), this problem is of the form $\phi(m) :=$

$$\overbrace{\mu_1(\psi_{\mathsf{init}})}^{\substack{\text{initial states}\\ \text{Line 1}}} \wedge \bigwedge_{\underbrace{j=1}_{\substack{\text{one conjunct}\\ \text{per step}}}}^{b(m)} \left(\underbrace{(\overset{(j)}{x}_{\mathsf{id}} \doteq 1 \vee \overset{(j)}{x}_{\mathsf{id}} \neq \overset{(j-1)}{x}_{\mathsf{id}})}_{\substack{\text{transitivity}\\ \text{Line 7}}} \wedge \underbrace{\bigvee_{i=1}^{\ell'(m)} \mu_j(\pi_i \wedge x_{\mathsf{id}} \doteq i)}_{\substack{\text{transition relation}\\ \text{Line 8}}} \wedge \underbrace{\bigwedge_{(j,\pi)\in \textsc{blocked}(m)} \pi}_{\substack{\text{blocking clauses}\\ \text{Line 9}}} \right)$$

where $b(m)$, $\ell'(m)$, and $\textsc{blocked}(m)$ are the value of b, the length of $\boldsymbol{\pi}$, and the values of $\textsc{blocked}$ in Line 10 in the mth iteration, respectively. For simplicity, here we use an additional variable $\overset{(0)}{x}_{\mathsf{id}}$ which only occurs in the first transitivity constraint (which is not generated by Alg. 2):

$$\overset{(1)}{x}_{\mathsf{id}} \doteq 1 \vee \overset{(1)}{x}_{\mathsf{id}} \neq \overset{(0)}{x}_{\mathsf{id}}$$

Therefore, this constraint is trivially satisfiable (e.g., by setting $\overset{(0)}{x}_{\mathsf{id}}$ to 1).

Assume that $\mathcal{T}$ is unsafe, but Alg. 2 returns safe. Then there is some $k \in \mathbb{N}$ such that $\phi(k)$ is unsatisfiable, and $\phi(k')$ is satisfiable for all $1 \leq k' < k$ (otherwise, Alg. 2 would have returned safe in an earlier iteration).

As Alg. 2 backtracks in Line 19, we consider the sequence of natural numbers $1 \leq i_1 < \ldots < i_{b(k)} = k$ such that for all $1 \leq c \leq b(k)$, i_c is the last iteration where $b = c$ in Line 10. Then for all $1 \leq B \leq b(k)$, we have $\phi(i_B) = \varphi(B)$ where

$$\varphi(B) := \mu_1(\psi_{\mathsf{init}}) \wedge \bigwedge_{j=1}^{B} \left((\overset{(j)}{x}_{\mathsf{id}} \doteq 1 \vee \overset{(j)}{x}_{\mathsf{id}} \neq \overset{(j-1)}{x}_{\mathsf{id}}) \wedge \bigvee_{i=1}^{\ell(B)} \mu_j(\pi_i \wedge x_{\mathsf{id}} \doteq i) \wedge \bigwedge_{(j,\pi)\in \textsc{blocked}(k)} \pi \right).$$

Here, we have $\ell(B) := \ell'(i_B)$. A blocking clause π is only added to the SMT encoding if $(j, \pi) \in \textsc{blocked}$ and $b = j$ (see Line 9) and Alg. 2 backtracks until $b \leq j$ whenever such an element is added to $\textsc{blocked}$ (see Line 19). So when only considering the iterations $i_1, \ldots, i_{b(k)}$, then $b(i_B) = B$ and all blocking clauses of the form $(j, \pi) \in \textsc{blocked}(k)$ where $j < B$ are already present when unrolling the transition relation for the Bth time, i.e., they are already contained in $\textsc{blocked}(i_B)$.

In other words, we have

$$\{(j, \pi) \mid (j, \pi) \in \textsc{blocked}(k) \mid j < B\} \subseteq \textsc{blocked}(i_B).$$

Thus, we may use $\textsc{blocked}(k)$ instead of $\textsc{blocked}(i_B)$ in the definition of φ.

Let $c \in \mathbb{N}$ and $\mathbf{v}_0, \ldots \mathbf{v}_c \in \mathcal{C}^d$ be arbitrary but fixed where $[\mathbf{x}/\mathbf{v}_0] \models \psi_{\mathsf{init}}$ and

$$\mathbf{v}_0 \to_\tau \ldots \to_\tau \mathbf{v}_c.$$

We use induction on c to show that[10]

$$\forall 0 \leq i \leq c.\ \exists B(i) < b(k), h(0,i) < \ldots < h(B(i),i).\ h(0,i) = 0 \wedge h(B(i),i) = i \\ \wedge \varphi(B(i)) \text{ is consistent with } [\mu_{j+1}(\mathbf{x})/\mathbf{v}_{h(j,i)} \mid 0 \leq j \leq B(i)]. \quad (8)$$

Intuitively, $B(i)$ is the number of steps that are needed to reach $\mathbf{v}_i$ when also using learned relations, and $\mathbf{v}_{h(j,i)}$ is the jth state in the resulting run that leads to $\mathbf{v}_i$. Thus, (8) shows that for all (arbitrary long) runs that start in a state satisfying ψ_{init}, all states that are reachable with $\rightarrow_\tau$ in arbitrarily many steps can also be reached in less than $b(k)$ steps (i.e., in constantly many steps) if one may also use the (transitive) learned relations. Of course, this only holds provided that Alg. 2 returns safe in the kth iteration.

Once we have shown (8), we can prove the theorem: We had assumed that $\mathcal{T}$ is unsafe, i.e., that there is a reachable error state $\mathbf{v}_c$ and that $\phi(k) = \phi(i_{b(k)}) = \varphi(b(k))$ is unsatisfiable. We use (8) for $i = c$: By (8), there is some $B(c) < b(k)$ such that

$$\varphi(B(c)) \text{ is consistent with } [\mu_{j+1}(\mathbf{x})/\mathbf{v}_{h(j,c)} \mid 0 \leq j \leq B(c)].$$

Moreover, as $\mathbf{v}_c$ is an error state, ψ_{err} is consistent with $[\mathbf{x}/\mathbf{v}_c]$ and hence,

$$\mu_{B(c)+1}(\psi_{\mathsf{err}}) \text{ is consistent with } [\mu_{B(c)+1}(\mathbf{x})/\mathbf{v}_c] = [\mu_{B(c)+1}(\mathbf{x})/\mathbf{v}_{h(B(c),c)}].$$

Thus,

$$\varphi(B(c)) \wedge \mu_{B(c)+1}(\psi_{\mathsf{err}}) \text{ is consistent with } [\mu_{j+1}(\mathbf{x})/\mathbf{v}_{h(j,c)} \mid 0 \leq j \leq B(c)].$$

Hence,

$$\text{Alg. 2 returns unknown in Line 5 in iteration } i_{B(c)+1} \quad (9)$$

or earlier. The reason is that we have $b = B(c) + 1$ in iteration $i_{B(c)+1}$, so in this iteration Alg. 2 checks satisfiability of the formula $\varphi(B(c)) \wedge \mu_{B(c)+1}(\psi_{\mathsf{err}})$ in Line 5. As we have $b(k) > B(c)$, we get $k = i_{b(k)} \geq i_{B(c)+1}$. Hence, (9) contradicts the assumption that Alg. 2 returns safe in Line 10 in iteration k.

We now prove (8). In the induction base, we have

$$\begin{aligned} & [\mathbf{x}/\mathbf{v}_0] \models \psi_{\mathsf{init}} \\ \curvearrowright\ & [\mu_1(\mathbf{x})/\mathbf{v}_0] \models \mu_1(\psi_{\mathsf{init}}) \\ \curvearrowright\ & [\mu_1(\mathbf{x})/\mathbf{v}_0] \models \varphi(0) && (\text{as } \varphi(0) \equiv \mu_1(\psi_{\mathsf{init}})) \end{aligned}$$

Hence, the claim follows for

$$B(0) = 0 = h(0,0).$$

In the induction step, the induction hypothesis implies

$$\forall 0 \leq i < c.\ \exists B(i) < b(k), h(0,i) < \ldots < h(B(i),i).\ h(0,i) = 0 \wedge h(B(i),i) = i \\ \wedge \varphi(B(i)) \text{ is consistent with } [\mu_{j+1}(\mathbf{x})/\mathbf{v}_{h(j,i)} \mid 0 \leq j \leq B(i)]. \quad (10)$$

Let:

$$I := \min\{i \mid 0 \leq i < c, 1 \leq j \leq \ell(B(i)+1), \mathbf{v}_i \rightarrow_{\pi_j} \mathbf{v}_c\}$$
$$J := \max\{j \mid 1 \leq j \leq \ell(B(I)+1), \mathbf{v}_I \rightarrow_{\pi_j} \mathbf{v}_c\}$$

So I is the minimal index such that we can make a step from $\mathbf{v}_I$ to $\mathbf{v}_c$, and among the relational formulas that can be used for this step, π_J is the one that was learned last. Note that I (and hence also J) exists, as we have $\mathbf{v}_{c-1} \rightarrow_\tau \mathbf{v}_c$ and $\tau = \pi_1$. By (10),

$$\varphi(B(I)) \text{ is consistent with } [\mu_{j+1}(\mathbf{x})/\mathbf{v}_{h(j,I)} \mid 0 \leq j \leq B(I)].$$

[10] While $\varphi(B)$ only corresponds to a formula that is checked by Alg. 2 if $B > 0$, $\varphi(0) \equiv \mu_1(\psi_{\mathsf{init}})$ is well defined, too.

Let θ_I be an extension of $[\mu_{j+1}(\mathbf{x})/\mathbf{v}_{h(j,I)} \mid 0 \leq j \leq B(I)]$ such that $\theta_I \models \varphi(B(I))$, and let θ be an extension of

$$\theta_I \uplus [\mu_{B(I)+2}(\mathbf{x})/\mathbf{v}_c] \uplus [\mu_{B(I)+1}(x_{\mathsf{id}})/J] \tag{11}$$

such that $\theta \models \mu_{B(I)+1}(\pi_J)$, which exists as we have:

$$\begin{aligned}
&\theta(\mu_{B(I)+1}(\mathbf{x})) \\
&= \theta_I(\mu_{B(I)+1}(\mathbf{x})) && \text{(by (11))} \\
&= \mathbf{v}_{h(B(I),I)} && \text{(def. of } \theta_I\text{)} \\
&= \mathbf{v}_I && \text{(as } h(B(I),I) = I\text{)} \\
&\to_{\pi_J} \mathbf{v}_c && \text{(def. of } J\text{)} \\
&= \mu_{B(I)+1}(\mathbf{x}')[\mu_{B(I)+1}(\mathbf{x}')/\mathbf{v}_c] \\
&= \mu_{B(I)+1}(\mathbf{x}')[\mu_{B(I)+2}(\mathbf{x})/\mathbf{v}_c] && \text{(as } \mu_{B(I)+2}(\mathbf{x}) = \mu_{B(I)+1}(\mathbf{x}')\text{)} \\
&= \theta(\mu_{B(I)+1}(\mathbf{x}')) && \text{(by (11))}
\end{aligned}$$

So we have

$$\theta({}^{(B(I)+1)}\mathbf{x}) = \theta_I({}^{(B(I)+1)}\mathbf{x}) = \mathbf{v}_{h(B(I),I)} = \mathbf{v}_I$$

and

$$\theta({}^{(B(I)+2)}\mathbf{x}) = \mathbf{v}_c.$$

We now show $\theta \models \varphi(B(I)+1)$. Then we get $B(c) = B(I)+1$, $h(j,c) = h(j,I)$ for all $j \leq B(I)$, and $h(B(c),c) = c$, which finishes the proof of (8).

Since θ is an extension of θ_I, we have $\theta \models \varphi(B(I))$, so we only need to show

$$\theta \models ({}^{(B(I)+1)}x_{\mathsf{id}} \doteq 1 \vee {}^{(B(I)+1)}x_{\mathsf{id}} \not\doteq {}^{(B(I))}x_{\mathsf{id}}) \wedge \bigvee_{i=1}^{\ell(B(I)+1)} \mu_{B(I)+1}(\pi_i \wedge x_{\mathsf{id}} \doteq i) \wedge \bigwedge_{(B(I)+1,\pi)\in \textsc{blocked}(k)} \pi,$$

i.e., we only need to show that θ is a model of the last conjunct of $\varphi(B(I)+1)$. For the disjunction

$$\bigvee_{i=1}^{\ell(B(I)+1)} \mu_{B(I)+1}(\pi_i \wedge x_{\mathsf{id}} \doteq i), \tag{12}$$

note that $J \leq \ell(B(I)+1)$. Thus, to show $\theta \models$ (12), it suffices if

$$\theta \models \mu_{B(I)+1}(\pi_J \wedge x_{\mathsf{id}} \doteq J),$$

which holds by construction of θ. Thus, it remains to show

$$\theta \models ({}^{(B(I)+1)}x_{\mathsf{id}} \doteq 1 \vee {}^{(B(I)+1)}x_{\mathsf{id}} \not\doteq {}^{(B(I))}x_{\mathsf{id}}) \wedge \bigwedge_{(B(I)+1,\pi)\in \textsc{blocked}(k)} \pi.$$

We first consider the disjunction ${}^{(B(I)+1)}x_{\mathsf{id}} \doteq 1 \vee {}^{(B(I)+1)}x_{\mathsf{id}} \not\doteq {}^{(B(I))}x_{\mathsf{id}}$. We show that we always have $J = 1$ or $\theta({}^{(B(I))}x_{\mathsf{id}}) \neq J$. Then this disjunction is clearly satisfied by θ since $\theta({}^{(B(I)+1)}x_{\mathsf{id}}) = J$. To see why $J = 1$ or $\theta({}^{(B(I))}x_{\mathsf{id}}) \neq J$ holds, assume that $\theta({}^{(B(I))}x_{\mathsf{id}}) = J > 1$. Then:

$$\begin{aligned}
&\theta \models \varphi(B(I)) && \text{(as } \theta \text{ is an extension of } \theta_I\text{)} \\
&\curvearrowright \theta \models \mu_{B(I)}(\pi_J \wedge x_{\mathsf{id}} \doteq J) && \text{(as } \theta({}^{(B(I))}x_{\mathsf{id}}) = J \text{ by assumption)} \\
&\curvearrowright \mu_{B(I)}(\pi_J) \text{ is consistent with } [\mu_{j+1}(\mathbf{x})/\mathbf{v}_{h(j,I)} \mid 0 \leq j \leq B(I)] \\
& && \text{(as } [\mu_{j+1}(\mathbf{x})/\mathbf{v}_{h(j,I)} \mid 0 \leq j \leq B(I)] \subseteq \theta\text{)}
\end{aligned}$$

$\curvearrowright \mu_{B(I)}(\pi_J)$ is consistent with $[\mu_{B(I)}(\mathbf{x})/\mathbf{v}_{h(B(I)-1,I)}, \mu_{B(I)+1}(\mathbf{x})/\mathbf{v}_{h(B(I),I)}]$
(by instantiating j with $B(I)-1$ and $B(I)$)

$\curvearrowright \pi_J$ is consistent with $[\mathbf{x}/\mathbf{v}_{h(B(I)-1,I)}, \mathbf{x}'/\mathbf{v}_{h(B(I),I)}]$

$\curvearrowright \pi_J$ is consistent with $[\mathbf{x}/\mathbf{v}_{h(B(I)-1,I)}, \mathbf{x}'/\mathbf{v}_I]$ (as $h(B(I),I) = I$)

$\curvearrowright \mathbf{v}_{h(B(I)-1,I)} \rightarrow_{\pi_J} \mathbf{v}_I$.

By the definition of I and J, we also have $\mathbf{v}_I \rightarrow_{\pi_J} \mathbf{v}_c$. So by transitivity of $\rightarrow_{\pi_J}$ (which holds since $J > 1$), we get $\mathbf{v}_{h(B(I)-1,I)} \rightarrow_{\pi_J} \mathbf{v}_c$. As we have $h(B(I)-1, I) < I$ by definition of h, this contradicts minimality of I. Note that here is the only point where we need the transitivity of learned relations.

Therefore, it remains to show

$$\theta \models \bigwedge_{(B(I)+1,\pi)\in \textsc{blocked}(k)} \pi.$$

The elements of $\textsc{blocked}(k)$ have the form $(s+\ell-1, \mathsf{blocking_clause}(s, \ell, \pi_j, \overline{\sigma}))$ where $[\tau_i]_{i=s}^{s+\ell-1}$ is a loop on the trace, $\pi_j \in \mathsf{tail}(\boldsymbol{\pi})$ is a learned relation, and $\overline{\sigma} \models \pi_j$. Moreover, we have $s+\ell-1 = B(I)+1$. We now perform a case analysis for the two cases where $\ell = 1$ and $\ell > 1$.

In Case 1 (where $\ell = 1$), the blocking clause has the form $\mathsf{blocking_clause}(s, 1, \pi_j, \overline{\sigma})$ for $s = B(I)+1$. Thus, we show that θ cannot violate a blocking clause of the form

$$\mathsf{blocking_clause}(B(I), 1, \pi_j, \overline{\sigma}) = \mu_{B(I)+1}(\neg \mathsf{cvp}(\pi_j, \overline{\sigma})) \vee {}^{(B(I)+1)}x_{\mathsf{id}} > 1.$$

To see this, we first consider the case where the negation of the first disjunct holds and prove that then the second disjunct is true. The reason is that we have:

$\theta \models \mu_{B(I)+1}(\mathsf{cvp}(\pi_j, \overline{\sigma}))$

$\curvearrowright \theta \circ \mu_{B(I)+1}$ is consistent with π_j (since $\mathsf{cvp}(\pi_j, \overline{\sigma}) \models \pi_j$ by Def. 2)

$\curvearrowright \theta(\mu_{B(I)+1}(\mathbf{x})) \rightarrow_{\pi_j} \theta(\mu_{B(I)+1}(\mathbf{x}'))$

$\curvearrowright \theta(\mu_{B(I)+1}(\mathbf{x})) \rightarrow_{\pi_j} \theta(\mu_{B(I)+2}(\mathbf{x}))$

$\curvearrowright \mathbf{v}_{h(B(I),I)} \rightarrow_{\pi_j} \mathbf{v}_c$ (def. of θ)

$\curvearrowright \mathbf{v}_I \rightarrow_{\pi_j} \mathbf{v}_c$ (as $h(B(I),I) = I$)

So the step from $\mathbf{v}_I$ to $\mathbf{v}_c$ can be done by a learned relation π_j. As π_j is a learned relation, we have $j > 1$. This implies $\theta({}^{(B(I)+1)}x_{\mathsf{id}}) = J > 1$, as J is maximal and hence $J \geq j > 1$.

Now we consider the case where the negation of the first disjunct does not hold and prove that then the first disjunct is true. The reason is that due to the completeness of $\mathcal{A}$, $\theta \not\models \mu_{B(I)+1}(\mathsf{cvp}(\pi_j, \overline{\sigma}))$ implies $\theta \models \mu_{B(I)+1}(\neg \mathsf{cvp}(\pi_j, \overline{\sigma}))$. So in this case the blocking clause is satisfied as well.

In Case 2 (where $\ell > 1$), we show that θ cannot violate a blocking clause of the form

$$\mathsf{blocking_clause}(s, \ell, \pi_j, \overline{\sigma}) = \mu_{s,\ell}(\neg \mathsf{cvp}(\pi_j, \overline{\sigma}))$$

where $s+\ell-1 = B(I)+1$, i.e., $s+\ell = B(I)+2$. The reason is that we have

$\theta \not\models \mu_{s,\ell}(\neg \mathsf{cvp}(\pi_j, \overline{\sigma}))$

$\curvearrowright \theta \models \mu_{s,\ell}(\mathsf{cvp}(\pi_j, \overline{\sigma}))$ (as $\mathcal{A}$ is complete)

$\curvearrowright \theta \circ \mu_{s,\ell}$ is consistent with π_j (since $\mathsf{cvp}(\pi_j, \overline{\sigma}) \models \pi_j$ by Def. 2)

$\curvearrowright \theta(\mu_{s,\ell}(\mathbf{x})) \rightarrow_{\pi_j} \theta(\mu_{s,\ell}(\mathbf{x}'))$

$\curvearrowright \mathbf{v}_{h(s-1,I)} \rightarrow_{\pi_j} \mathbf{v}_c$ (def. of θ, as $s+\ell = B(I)+2$)

For the last step, note that

$$\theta(\mu_{s,\ell}(\mathbf{x})) = \theta(\mu_s(\mathbf{x})) = \theta_I(\mu_s(\mathbf{x})) = \mathbf{v}_{h(s-1,I)} \qquad \text{and}$$
$$\theta(\mu_{s,\ell}(\mathbf{x}')) = \theta(\overset{(s+\ell)}{\mathbf{x}}) = \theta(\overset{(B(I)+2)}{\mathbf{x}}) = \mathbf{v}_c.$$

We have $s-1 < B(I)$ and thus $h(s-1,I) < h(B(I),I) = I$, which contradicts minimality of I. This finishes the proof of (8). □

A.2 Proof of Thm. 28

Theorem 28 (Correctness of tp)**.** *The function* tp *as defined in Sect. 4 is a transitive projection.*

Proof We have to prove that

(a) $\mathsf{tp}(\tau,\sigma)$ is consistent with σ,

(b) $\{\mathsf{tp}(\tau,\theta) \mid \theta \models \tau\}$ is finite, and

(c) $\to_{\mathsf{tp}(\tau,\sigma)}$ is transitive

for all transitions τ and all $\sigma \models \tau$.

For item (a), we first prove $\sigma \models \tau_{\mathsf{rec}}$. We have:

$$\begin{aligned}
&\sigma \models \tau \\
\curvearrowright\ &\sigma \uplus [x_\delta/\sigma(x'-x) \mid x \in \mathbf{x}] \models \tau_{\wedge\delta} && \text{(by def. of } \tau_{\wedge\delta}\text{)} \\
\curvearrowright\ &[x_\delta/\sigma(x'-x) \mid x \in \mathbf{x}] \models \tau_\delta && \text{(by def. of } \mathsf{cvp} \text{ and } \tau_\delta\text{)} \\
\curvearrowright\ &[x/\sigma(x), x'/\sigma(x') \mid x \in \mathbf{x}] \models \tau_{\mathsf{rec}} && \text{(by def. of } \tau_{\mathsf{rec}}\text{)} \\
\curvearrowright\ &\sigma \models \tau_{\mathsf{rec}}
\end{aligned}$$

Now we prove $\sigma' := \sigma \uplus [n/1] \models \mathsf{tp}(\tau,\sigma)$. To this end, we consider the literals that are added to $\mathsf{tp}(\tau,\sigma)$ by the procedure described in Sect. 4 independently:

- $\sigma' \models n > 0$ is trivial
- if $\sigma \models (\sum_{x\in\mathbf{x}} c_x \cdot (x'-x) + c) \bowtie 0$, then $\sigma' \models (\sum_{x\in\mathbf{x}} c_x \cdot (x'-x) + n\cdot c) \bowtie 0$
- if $\sigma \models e \mid (\sum_{x\in\mathbf{x}} c_x \cdot (x'-x) + c)$, then $\sigma' \models e \mid (\sum_{x\in\mathbf{x}} c_x \cdot (x'-x) + n\cdot c)$

Moreover, we have $\sigma \models \mathsf{cvp}(\tau,\sigma,\mathbf{x})$ and $\sigma \models \mathsf{cvp}(\tau,\sigma,\mathbf{x}')$ by definition of cvp, and hence we also have $\sigma' \models \mathsf{cvp}(\tau,\sigma,\mathbf{x})$ and $\sigma' \models \mathsf{cvp}(\tau,\sigma,\mathbf{x}')$. Therefore, σ' is a model of $\mathsf{tp}(\tau,\sigma)$, so σ is consistent with $\mathsf{tp}(\tau,\sigma)$.

For item (b), note that tp only uses the provided model for computing conjunctive variable projections. As cvp has a finite image, the claim follows.

For item (c), note that the conjuncts $\mathsf{cvp}(\tau,\sigma,\mathbf{x})$ and $\mathsf{cvp}(\tau,\sigma,\mathbf{x}')$ of $\mathsf{tp}(\tau,\sigma)$ are irrelevant for transitivity, as they do not relate $\mathbf{x}$ and $\mathbf{x}'$. Let $\tau' := \mathsf{tp}(\tau,\sigma)$. It suffices to prove

$$\to_{\tau'}^2 \subseteq \to_{\tau'}.$$

Then the claim follows from a straightforward induction. Assume $\theta \models \tau'$, $\theta' \models \tau'$, and

$$\theta(\mathbf{x}) \to_{\tau'} \theta(\mathbf{x}') = \theta'(\mathbf{x}) \to \theta'(\mathbf{x}').$$

We prove $\hat{\theta} \models \tau'$ where

$$\hat{\theta} := [\mathbf{x}/\theta(\mathbf{x})] \uplus [\mathbf{x}'/\theta'(\mathbf{x}')] \uplus [n/\theta(n) + \theta'(n)].$$

Then the claim follows. Again, we consider all literals independently:

- If $\theta \models n > 0$ and $\theta' \models n > 0$, then $\hat{\theta} \models n > 0$.
- Consider a literal of the form $\iota := t \bowtie 0$ where $t = \sum_{x \in \mathbf{x}} c_x \cdot (x' - x) + n \cdot c$. We have

$$\begin{aligned}
\theta(t) &= \sum_{x \in \mathbf{x}} c_x \cdot (\theta(x') - \theta(x)) + \theta(n) \cdot c \\
&= \sum_{x \in \mathbf{x}} c_x \cdot \theta(x') - \sum_{x \in \mathbf{x}} c_x \cdot \theta(x) + \theta(n) \cdot c \\
&= \sum_{x \in \mathbf{x}} c_x \cdot \theta'(x) - \sum_{x \in \mathbf{x}} c_x \cdot \theta(x) + \theta(n) \cdot c \qquad (\text{as } \theta(\mathbf{x}') = \theta'(\mathbf{x}))
\end{aligned}$$

  and

$$\begin{aligned}
\theta'(t) &= \sum_{x \in \mathbf{x}} c_x \cdot (\theta'(x') - \theta'(x)) + \theta'(n) \cdot c \\
&= \sum_{x \in \mathbf{x}} c_x \cdot \theta'(x') - \sum_{x \in \mathbf{x}} c_x \cdot \theta'(x) + \theta'(n) \cdot c.
\end{aligned}$$

  Thus, we have:

$$\begin{aligned}
\theta(t) + \theta'(t) &= -\sum_{x \in \mathbf{x}} c_x \cdot \theta(x) + \theta(n) \cdot c + \sum_{x \in \mathbf{x}} c_x \cdot \theta'(x') + \theta'(n) \cdot c \\
&= \sum_{x \in \mathbf{x}} c_x \cdot \theta'(x') - \sum_{x \in \mathbf{x}} c_x \cdot \theta(x) + (\theta(n) + \theta'(n)) \cdot c \\
&= \sum_{x \in \mathbf{x}} c_x \cdot \hat{\theta}(x') - \sum_{x \in \mathbf{x}} c_x \cdot \hat{\theta}(x) + \hat{\theta}(n) \cdot c \\
&= \hat{\theta}(t)
\end{aligned}$$

  Therefore, $\theta \models \iota$ and $\theta' \models \iota$ implies $\hat{\theta} \models \iota$ for all $\bowtie \in \{\leq, \geq, <, >, =\}$.
- Consider a literal of the form $\iota := e|t$ where $t = \sum_{x \in \mathbf{x}} c_x \cdot (x' - x) + n \cdot c$. Then we again obtain

$$\theta(t) + \theta'(t) = \hat{\theta}(t)$$

  as above. Thus, $\theta \models \iota$ and $\theta' \models \iota$ imply $\hat{\theta} \models \iota$.

□